

**DOSEMATE: A DIGITAL REMINDER SOLUTION FOR ADVANCING
MEDICAL ADHERENCE THROUGH SMS AUTOMATION USING RULE-
BASED SCHEDULING, TRIGGERING, AND SCHEDULED EXPIRY
PURGE ALGORITHM**

A PROJECT REPORT

(PHASE I)

Submitted by

SHREE HARINESH S. (Register No: 727822TUCS216)

VIKRAM P. (Register No: 727822TUCS253)

YUVARAJ VC. (Register No: 727822TUCS261)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



SRI KRISHNA COLLEGE OF TECHNOLOGY
An Autonomous Institution | Accredited by NAAC with 'A' Grade
Affiliated to Anna University | Approved by AICTE
KOVAIPUDUR, COIMBATORE 641042



NOVEMBER 2025



SRI KRISHNA COLLEGE OF TECHNOLOGY
An Autonomous Institution | Accredited by NAAC with 'A'
Grade Affiliated to Anna University | Approved by AICTE
KOVAIPUDUR, COIMBATORE 641042



BONAFIDE CERTIFICATE

Certified that this project report **“DOSEMATE: A DIGITAL REMINDER SOLUTION FOR ADVANCING MEDICAL ADHERENCE THROUGH SMS AUTOMATION USING RULE-BASED SCHEDULING, TRIGGERING, AND SCHEDULED EXPIRY PURGE ALGORITHM”** is the bonafide work of **“SHREE HARINESH S., VIKRAM P., YUVARAJ V C.”** who carried out the project work phase I under my supervision.

SIGNATURE

Ms. A GOMATHY

SUPERVISOR

Assistant Professor,
Department of Computer Science
and Engineering,
Sri Krishna College of Technology,
Technology, Coimbatore-641042.

SIGNATURE

Dr. M. UDHAYAMOORTHY

HEAD OF THE DEPARTMENT

Associate Professor,
Department of Computer Science
and Engineering,
Sri Krishna College of
Coimbatore-641042.

Certified that the candidates were examined by us in the Project Phase I Viva Voce examination held on _____ at Sri Krishna College of Technology, Kovaipudur, Coimbatore -641042

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Medication non-adherence is a pressing global health issue that severely reduces treatment effectiveness and contributes to rising healthcare costs, with the World Health Organization estimating non-compliance rates of up to seventy percent in developing countries. This results in poor disease management, frequent hospitalizations, and billions of dollars lost annually due to preventable complications. Existing reminder applications are often either disease-specific or restricted by geography, making them unsuitable for large-scale deployment. To address this limitation, this paper presents DoseMate, a smart doctor-assisted prescription reminder system designed to ensure timely medication intake while enabling doctors to monitor patient compliance. The system is developed using a cloud-hosted Spring Boot backend, a React.js frontend, and the Twilio SMS gateway for global delivery of reminders. Its functionality is driven by three algorithms—Rule-Based Scheduling, Session-Time Triggering, and Scheduled Expiry Purge—which together ensure accurate, timely notifications that are automatically discontinued once treatment ends. Initial implementation and testing confirm that the system achieves high reliability in delivering reminders, demonstrating its potential as a globally scalable, disease-agnostic solution to improve adherence, reduce costs, and strengthen doctor-patient coordination.

ACKNOWLEDGEMENT

First and foremost we thank the **Almighty** for being our light and for showering his gracious blessings throughout the course of this project.

We express our gratitude to our beloved Principal, **Dr. M.G. Sumithra**, for providing all facilities.

With the grateful heart, our sincere thanks to our Head of the Department **Dr. M. Udhayamoorthi**, Department of Computer Science and Engineering for the motivation and all support to complete the project work.

We thank **Dr. M. Kavitha Margret**, Department of Computer Science and Engineering, Project Coordinator for her motivation and support

We extend our sincere thanks to our project supervisor **Ms. A. Gomathy**, Department of Computer Science and Engineering, for the valuable guidance and suggestions in all aspects that aided us to ameliorate our skills.

We are thankful to all the **Teaching and Non-Teaching Staff** of Department of Computer Science and Engineering and to all those who have directly and indirectly extended their help to us in completing this project work successfully.

We extend our sincere thanks to our **family members** and our beloved **friends**, who had been strongly supporting us in all our endeavours.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1.1	EXISTING SYSTEM	10
3.2.1	PROPOSED SYSTEM	12
4.1.1	FLOW DIAGRAM	14
5.3.1	NEW PATIENT	28
5.3.2	RETURNING PATIENT SMS	28
5.3.3	DIGITAL PRESCRIPTION	29
5.3.4	SESSION MEDICINE REMINDER	29
5.3.5	TEST REPORT REMINDER	30
5.3.6	EXPIRED REMINDER CLEANUP	30
5.3.7	ADMIN LOGIN	30
5.3.8	DOCTOR LOGIN	31
5.3.9	DOCTOR REGISTRATION	31
5.3.10	ADMIN DASHBOARD	32
5.3.11	MANAGE DOCTORS	32
5.3.12	DOCTOR DASHBOARD	33
5.3.13	PATIENT ADDITION	33
5.3.14	PATIENT OVERVIEW	34
6.1.1	SENDER POOL	68

6.1.2	MESSAGE LOGS	68
6.1.3	PATIENT DATA	69
6.1.4	MEDICINE REMINDER SCHEDULER	69
6.1.5	LAB REMINDER SCHEDULER	69

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
SMS	Short Message Service
API	Application Programmable Interface
DB	Database
UI	User Interface
HTTP	HyperText Transfer Protocol
CRUD	Create , Read , Update , Delete
MVC	Model – View - Controller
IDE	Integrated Development Environment
JSON	Javascript Object Notation

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	iii
	LIST OF ABBREVIATIONS	iv
1	INTRODUCTION	1
	1.1 DOSEMATE IN HEATHCARE REMINDER SYSTEM	1
	1.2 OBJECTIVE	2
2	LITERATURE SURVEY	3
	2.1 MEDICATION NON-ADHERENCE AND GLOBAL HEALTH	3
	2.2 DIGITAL REMINDER SYSTEM	4
	2.3 CHALLENGES IN PATIENT ADHERENCE	5
	2.4 ROLE OF AUTOMATED SMS REMINDER	5
	2.5 PATIENT AND DOCTOR ENGAGEMENT	7
	2.6 MOTIVATION	8

3	SYSTEM ANALYSIS OF SUPPLY CHAIN MANAGEMENT	9
	3.1 EXISTING SYSTEM	9
	3.2 PROPOSED SYSTEM	11
4	SYSTEM ARCHITECTURE	13
	4.1 SYSTEM OVERVIEW	13
	4.2 BACKEND IMPLEMENTATION	15
	4.3 SMS REMINDER USING TWILIO	18
	4.4 FRONTEND IMPLEMENTATION	20
	4.5 SUMMARY AND INSIGHTS	22
5	IMPLEMENTATION AND RESULTS	24
	5.1 SYSTEM SPECIFICATION	24
	5.2 SYSTEM IMPLEMENTATION	24
	5.2.1 BACKEND SCHEDULING AND REMINDER	25
	5.2.2 FRONTEND INTERFACE USING REACT JS	26
	5.2.3 SMS DELIVERY USING TWILIO	27

	5.3 RESULTS	27
	5.3.1 OUTPUT	28
6	CONCLUSION	35
	6.1 CONCLUSION	35
	6.2 FUTURE WORK	35
	REFERENCES	36
	APPENDIX 1	38

CHAPTER 1

INTRODUCTION

1.1 DOSEMATE IN HEALTHCARE REMINDER SYSTEM

Medication non-adherence — the failure to take medicines as prescribed — is one of the most critical challenges in global healthcare today. It affects millions of patients across different age groups and medical conditions, resulting in delayed recovery, increased hospital readmissions, and even life-threatening complications. Studies have shown that a large percentage of patients forget to take their medication or skip doses due to busy schedules, forgetfulness, or a lack of proper monitoring. This not only compromises individual health but also places a significant financial burden on healthcare systems worldwide. While some existing reminder systems attempt to tackle this issue, many of them are designed for specific diseases such as diabetes or hypertension, or they operate only within limited geographical areas. As a result, a large portion of the global population remains underserved, lacking a reliable and universal solution that can adapt to varied healthcare needs. DoseMate has been developed to bridge this crucial gap by offering a comprehensive, intelligent, and automated healthcare reminder platform. The system is capable of sending timely SMS notifications to patients for a wide range of healthcare activities — including regular medication intake, scheduled injections, and diagnostic scan appointments. Through the integration of a cloud-hosted backend and real-time scheduling algorithms, DoseMate ensures that every reminder is personalized, accurate, and delivered exactly when needed. To achieve seamless communication, DoseMate utilizes Twilio's global messaging services, allowing it to send reminders to patients across different countries without geographical limitations. The platform is designed to be scalable and user-friendly, ensuring that healthcare providers can easily register patients, set medication schedules, and monitor adherence patterns through an intuitive interface.

1.2 OBJECTIVE

The primary objective of this project is to design and develop an intelligent, technology-driven healthcare reminder system that enhances patient adherence to prescribed treatments while simultaneously assisting doctors and healthcare providers in managing their schedules more effectively. One of the major challenges in modern healthcare is ensuring that patients follow their treatment plans consistently. Missed doses, delayed injections, or forgotten diagnostic appointments can significantly impact recovery and overall health outcomes. DoseMate aims to minimize these challenges by introducing a fully automated reminder mechanism that ensures patients are consistently informed and reminded of their medical routines. The system's core functionality revolves around the automation of SMS reminders for various healthcare needs such as medication intake, injections, and diagnostic scans. These reminders are generated based on doctor-defined schedules that are tailored to each patient's treatment plan. To ensure precision and reliability, the system employs intelligent algorithms including Rule-Based Scheduling, Session-Time Triggering, and Scheduled Expiry Purge.

- **The Rule-Based Scheduling algorithm** ensures that reminders are created according to medical prescriptions and timing rules set by the doctor.
- **The Session-Time Triggering algorithm** manages real-time delivery of messages, ensuring that patients receive reminders exactly when they are due.
- **The Scheduled Expiry Purge algorithm** maintains system efficiency by automatically deleting outdated or expired reminders, ensuring that only relevant and active notifications are processed. In addition to the reminder feature, the project also emphasizes the importance of data accuracy and system integrity.

DoseMate securely stores patient information, treatment schedules, and medication details in a cloud-hosted environment, ensuring that the data remains consistent, up-to-date, and easily accessible for healthcare professionals. By regularly purging expired reminders and maintaining optimized database performance, the system ensures both speed and reliability in operations.

CHAPTER 2

LITERATURE SURVEY

2.1 MEDICATION NON-ADHERENCE AND GLOBAL HEALTH

Medication non-adherence has been identified in numerous studies as one of the most persistent and detrimental issues affecting global healthcare systems. It refers to patients not following their prescribed medication regimens—either by missing doses, taking incorrect quantities, or discontinuing treatment prematurely. Research consistently shows that such behavior contributes to delayed recovery, increased hospitalizations, and preventable medical complications. According to several global health reports, non-adherence is particularly problematic among patients suffering from chronic illnesses such as diabetes, hypertension, and cardiovascular diseases, where long-term medication compliance is essential. Elderly individuals and patients residing in rural or underdeveloped regions are more prone to non-adherence due to cognitive decline, low health literacy, or lack of access to consistent medical support. Moreover, factors such as complex medication schedules, financial constraints, and side effects often discourage patients from taking their medicines as prescribed. This leads to deteriorating health conditions, which in turn increase healthcare costs for both families and healthcare systems. Researchers have emphasized that enhancing medication adherence is not only a matter of improving patient behavior but also involves strengthening the overall healthcare infrastructure. Hospitals and clinics face additional burdens from preventable readmissions and prolonged treatments, which could be minimized through systematic adherence monitoring. Consequently, there is a growing need for technology-driven, scalable, and globally accessible reminder systems that can provide customized, timely, and consistent support to patients. These systems must be capable of addressing the diverse needs of individuals across different age groups, regions, and medical conditions to effectively improve treatment outcomes and healthcare efficiency.

2.2 DIGITAL REMINDER SYSTEMS

Digital reminder systems have gained substantial attention in the field of healthcare technology as effective tools to improve patient adherence. These systems utilize digital platforms—such as mobile applications, wearable devices, or SMS notifications—to remind patients about their upcoming doses, injections, or diagnostic appointments. The core idea is to bridge the communication gap between patients and healthcare providers by providing real-time, automated, and easily accessible notifications. Many of these platforms allow healthcare professionals to create and assign reminders based on specific treatment schedules, ensuring that patients are consistently aware of their next medication or medical task. Some advanced applications even track whether the patient has confirmed the reminder or completed the task, giving doctors insights into adherence patterns. Studies have shown that patients who receive regular digital reminders exhibit significantly higher levels of medication compliance compared to those who rely solely on memory or manual tracking. However, despite these advantages, the literature also highlights several limitations. A majority of existing reminder systems are designed to cater to a specific disease category, such as diabetes or hypertension, thereby excluding patients with other conditions. Furthermore, many of these systems are region-specific, often limited to certain hospitals, healthcare networks, or national health programs. As a result, there is a strong need for scalable, disease-agnostic systems that can serve a wider population regardless of location or illness type.

Digital reminder systems, especially those that operate through SMS, are seen as an effective and inclusive solution because they do not depend on internet access or smartphones. Implementing such technology can ultimately improve treatment consistency, reduce medical errors, and significantly lower overall healthcare costs by preventing avoidable hospital visits and medical complications.

2.3 CHALLENGES IN PATIENT ADHERENCE

Despite the increasing use of digital health technologies, patient non-adherence continues to be a multifaceted challenge influenced by behavioral, environmental, and technological factors. Forgetfulness remains the most common reason for missed doses, especially among elderly patients. In many cases, patients fail to fully understand the importance of taking medicines at prescribed intervals, leading to poor health outcomes. Another major challenge lies in the lack of real-time monitoring tools for doctors. Once patients leave the clinic or hospital, healthcare providers often lose visibility into whether treatment instructions are being followed. This disconnect between patients and doctors can result in ineffective treatment management and preventable relapses. The complexity of managing multiple prescriptions—commonly known as polypharmacy—further worsens adherence issues. Patients dealing with multiple diseases often struggle to keep track of varying doses and schedules. For individuals in rural regions, limited access to mobile devices, poor network connectivity, and low digital literacy also pose significant barriers to adopting existing reminder systems. Studies consistently identify these issues as critical obstacles in achieving effective healthcare delivery. Addressing them requires the integration of automated, intelligent reminder systems capable of sending timely notifications, maintaining accuracy, and functioning even in low-resource environments. Effective adherence management through reliable technology not only improves patient recovery rates but also alleviates the overall pressure on healthcare institutions.

2.4 ROLE OF AUTOMATED SMS REMINDERS

Automated SMS reminders have emerged as one of the most practical and inclusive technological interventions for improving medication adherence. Unlike mobile applications that require internet access and user proficiency, SMS reminders are universally accessible across different types of mobile phones and regions.

The integration of backend systems ensures that these reminders are precisely scheduled using Rule-Based and Session-Time algorithms, which adapt dynamically to patient-specific schedules. Additionally, an automatic purge mechanism removes outdated or redundant reminders to keep the system optimized and free from unnecessary clutter. Such automation minimizes the risk of missed or delayed notifications, enhances system reliability, and significantly reduces the manual workload of healthcare professionals. Instead of individually following up with patients, doctors can rely on the automated platform to maintain consistent communication. Studies have reported that SMS-based reminders can increase patient compliance rates by 30–50%, especially among elderly patients who may otherwise forget or misunderstand instructions.

Moreover, SMS-based reminders play a vital role in fostering accountability, consistency, and awareness among patients, serving as a bridge between prescribed medical plans and actual patient behavior. When patients receive regular, personalized messages reminding them to take their medication or attend scheduled appointments, they are more likely to stay attentive to their health routines. These reminders act as gentle yet powerful motivators, reinforcing the importance of timely medication intake and adherence to medical instructions. Over time, this consistent engagement cultivates a sense of self-discipline and health responsibility, encouraging patients to take a more proactive and informed role in managing their own treatment. In addition to promoting individual accountability, SMS reminders also help patients develop a better understanding of their treatment schedules. For many, especially the elderly or those with multiple prescriptions, remembering exact timings and dosages can be challenging. Automated reminders eliminate this confusion by providing clear, time-sensitive prompts that guide patients step by step through their medication journey. This reduces the risk of missed or duplicate doses and significantly enhances medication accuracy and safety.

2.5 PATIENT AND DOCTOR ENGAGEMENT

Effective healthcare depends on continuous engagement between patients and doctors. When patients are actively involved in their treatment process and doctors have the tools to monitor adherence, overall treatment outcomes improve significantly. Modern studies emphasize the importance of two-way communication systems that enable doctors to send reminders, monitor compliance, and receive feedback from patients in real time.

Platforms that facilitate this type of engagement foster a sense of collaboration and accountability. For example, if a patient consistently misses doses, doctors can be alerted and take corrective actions, such as rescheduling doses or providing counseling. This proactive monitoring prevents the situation from escalating into serious health complications. Similarly, patients benefit from personalized care and the assurance that their doctor is actively monitoring their progress. Engagement also enhances trust and transparency within the patient-doctor relationship. Patients who receive continuous support are more likely to adhere to prescribed treatments, leading to fewer missed doses and better long-term outcomes. Additionally, the data collected through automated platforms can be analyzed to identify adherence patterns, helping doctors improve treatment plans and optimize healthcare delivery.

In essence, technology-driven engagement systems help close the communication gap between patients and healthcare professionals. This not only leads to improved adherence but also reduces hospital readmissions, minimizes complications, and contributes to a more efficient and patient-centered healthcare ecosystem.

2.6 MOTIVATION

The motivation behind the development of the DoseMate system stems from the persistent global challenge of medication non-adherence and the inefficiencies of existing reminder mechanisms. Observations from healthcare research and real-world clinical settings indicate that a significant number of patients continue to miss doses, resulting in prolonged recovery periods and severe health complications. Existing reminder applications are often fragmented, disease-specific, or limited to certain regions, leaving a large population unserved. This gap inspired the creation of a universal, automated, and intelligent reminder platform that could cater to diverse patient needs without geographical or technological barriers. DoseMate integrates modern web technologies, cloud computing, and Twilio's global SMS messaging infrastructure to ensure accessibility and reliability. The platform is designed to benefit a wide range of users, including elderly individuals, patients in rural areas, and healthcare professionals managing multiple patients simultaneously.

By providing automated, timely, and accurate reminders for medications, injections, and diagnostic procedures, DoseMate aims to reduce missed doses, improve adherence, and enhance health outcomes. Additionally, the system provides doctors with an intuitive dashboard for monitoring treatment compliance, helping them make informed decisions and adjustments in real time.

The ultimate motivation of this project lies in demonstrating how technological innovation can bridge the gap between patients and healthcare providers, improving communication, accountability, and overall efficiency within the healthcare system. DoseMate aspires to represent a meaningful step toward smarter, data-driven, and patient-centered healthcare management for the modern world.

CHAPTER 3

SYSTEM ANALYSIS OF SUPPLY CHAIN MANAGEMENT

3.1 EXISTING SYSTEM

The current healthcare reminder systems primarily depend on manual methods or basic mobile applications, which come with numerous limitations and challenges. In most cases, patients are either verbally instructed by their doctors or given written schedules to follow for their medications, injections, or diagnostic appointments. However, such manual reminders are highly unreliable, as many patients tend to forget to take their prescribed medications or attend scheduled diagnostic scans and follow-up appointments. This forgetfulness often results in delayed recovery, deterioration of health conditions, or even serious medical complications due to missed doses or treatments.

Moreover, once patients leave the hospital or clinic, doctors generally lack effective mechanisms to track whether their patients are adhering to the prescribed treatment plans. The absence of real-time feedback and tracking systems creates a major gap in post-consultation treatment management. This not only makes it difficult for doctors to monitor progress but also prevents them from intervening promptly when patients fail to comply with their medication schedules. As a result, overall treatment outcomes are compromised, and chronic disease management becomes far more complex and inefficient.

Existing reminder systems that do exist are often disease-specific, focusing only on conditions such as diabetes or hypertension, and are not adaptable to a wide range of medical needs. In addition, many of these systems are limited to specific regions or healthcare networks, thereby restricting their accessibility and usefulness to a larger, more diverse population. Manual follow-ups by healthcare workers or family members, although helpful, are time-consuming and not always reliable, especially for patients managing multiple medications. The dependence on patient memory significantly increases the likelihood of missed doses and incomplete treatment adherence.

Another key issue lies in the lack of real-time monitoring and automated communication between patients and healthcare providers. Without such integration, doctors remain unaware of non-adherence until complications arise. This lack of timely insights complicates patient management and often leads to avoidable hospital readmissions or health deterioration.

Furthermore, digital literacy remains a significant barrier, particularly among elderly patients and those living in rural or underdeveloped areas. Many of these individuals struggle to use smartphone-based reminder applications due to limited familiarity with technology, poor internet connectivity, or lack of access to modern devices. Consequently, these populations are often excluded from the benefits of existing digital reminder systems, further widening the healthcare accessibility gap.

Overall, the current healthcare reminder methods are fragmented, inefficient, and inaccessible to a large portion of the population. There is a critical need for an intelligent, inclusive, and automated healthcare reminder and tracking system that can ensure timely medication adherence, provide real-time updates to doctors, and support users across all demographics—especially those who are elderly, technologically inexperienced, or living in remote regions.

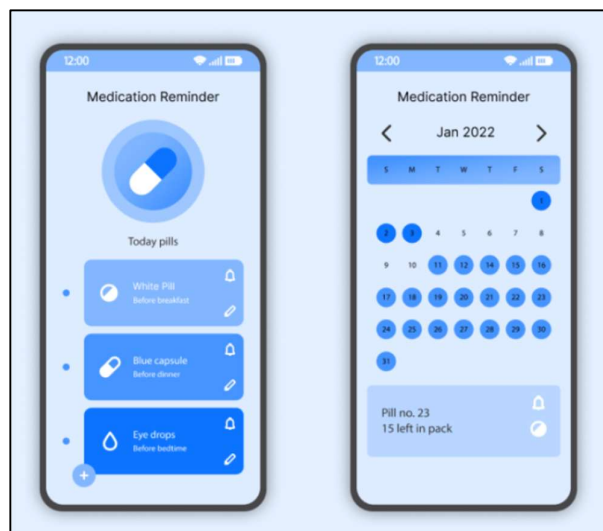


Fig 3.1.1 EXISTING SYSTEM

3.2 PROPOSED SYSTEM

In contrast to the limitations of existing healthcare reminder systems, the proposed DoseMate Healthcare Reminder System is designed to provide an intelligent, automated, and inclusive solution that effectively bridges the gap between patients and healthcare providers. This system leverages cloud-based technology integrated with Twilio's SMS notification service to deliver timely, reliable, and easily accessible reminders to patients. Instead of relying on manual follow-ups or mobile apps that require advanced user interaction, DoseMate simplifies the process by automatically sending SMS alerts for important medical activities such as taking medications, receiving injections, and attending diagnostic scans—all scheduled according to doctor-defined treatment plans.

The backbone of the system is powered by a robust Spring Boot backend, which handles the core automation processes including dynamic scheduling, session-based trigger management, and automatic purging of expired reminders. These features ensure that every reminder is delivered precisely when needed, without the need for manual oversight from hospital staff or patients. The automation not only increases reliability but also significantly reduces administrative workload for healthcare providers, allowing them to focus more on patient care rather than routine management tasks. The backend's efficient scheduling algorithms and database handling capabilities make the system both scalable and performance-optimized, capable of supporting large numbers of patients and reminders simultaneously.

A key strength of DoseMate lies in its real-time monitoring and analytics capabilities. Doctors can track each patient's adherence to prescribed treatments and medication schedules through the system's live dashboard. This immediate feedback allows healthcare professionals to quickly identify instances of non-compliance, missed doses, or delays in diagnostic follow-ups. With this insight, doctors can take proactive measures—such as sending personalized messages or adjusting treatment plans—to prevent potential health complications and improve overall treatment outcomes.

Furthermore, the system is designed with universal accessibility in mind. Unlike many modern healthcare apps that require constant internet access or

advanced smartphone usage, DoseMate ensures inclusivity by supporting patients who have limited access to smartphones or are not comfortable using mobile applications. By utilizing SMS as the primary communication method, the system ensures that even individuals in rural areas or those using basic feature phones can receive timely healthcare reminders. This makes DoseMate a truly global solution that transcends barriers of geography, technology, and digital literacy.

Another major advantage of DoseMate is its focus on data security, reliability, and system performance. All treatment-related information, such as medication schedules, doctor notes, and patient response data, is securely stored and managed on the cloud. The system automatically removes outdated reminders and redundant data to maintain efficiency and prevent database overload. This ensures that patients always receive accurate, up-to-date information while preserving their privacy and complying with healthcare data protection standards.

By combining automation, accessibility, and real-time monitoring, DoseMate not only minimizes the chances of missed doses or delayed treatments but also fosters stronger communication and trust between patients and healthcare professionals. Ultimately, the platform enhances treatment adherence, reduces preventable health complications, and streamlines healthcare delivery through an intelligent, secure, and user-friendly reminder management system.

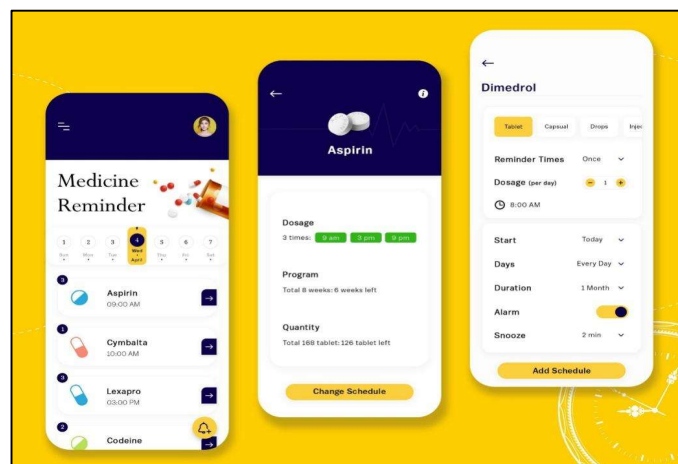


Fig 3.2.1 PROPOSED SYSTEM

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 SYSTEM OVERVIEW

The proposed DoseMate Healthcare Reminder System is an advanced, automated solution meticulously designed to enhance patient adherence, ensure timely medication intake, and streamline treatment scheduling for healthcare providers. The primary objective of the system is to assist doctors in managing patient treatment plans efficiently while helping patients stay consistent with their prescribed routines. Through an intuitive interface, doctors can conveniently enter detailed patient information, including personal details, diagnosis-related data, and customized treatment schedules that specify prescribed medications, injections, laboratory tests, and diagnostic scan appointments. This eliminates the need for manual tracking and minimizes the chances of missed or delayed treatments.

To ensure reminders are delivered at the most appropriate times, the system intelligently divides each day into four structured sessions—morning, afternoon, evening, and night. Within each session, the system sends two timely reminders, ensuring that patients are consistently notified throughout the day. This design helps cover various time zones and personal routines, providing flexibility and reliability. The scheduling of these reminders is handled automatically by the Reminder Scheduling Module, which calculates precise trigger times based on doctor-defined inputs such as dosage frequency, time intervals, and treatment duration. This level of automation removes the burden of manual scheduling for healthcare staff, significantly improving efficiency and consistency in patient care.

The SMS Notification Module, integrated through Twilio's cloud-based communication service, acts as the system's communication backbone. Once the Reminder Scheduling Module determines the appropriate timing, this module instantly sends out SMS alerts to patients, reminding them of upcoming doses, lab test appointments, diagnostic scans, or injections.

Unlike app-based notifications that require an internet connection or smartphone access, SMS reminders are universally accessible, ensuring even patients in remote or rural areas—and those without smartphones—can receive timely alerts. This makes DoseMate especially beneficial for elderly individuals and populations with limited digital literacy, offering a more inclusive and dependable healthcare reminder solution.

Another critical aspect of the system's architecture is its automated reminder management and performance optimization mechanism. To maintain high system performance and prevent database overload, DoseMate includes a daily automatic purging process that removes all expired reminders and outdated entries from the system. This ensures that only active, relevant reminders remain in circulation, reducing redundancy and maintaining fast, efficient system operations.

By combining these intelligent modules, the DoseMate system creates a fully automated, consistent, and reliable healthcare reminder framework that requires no manual follow-up from doctors or healthcare staff. The integration of real-time scheduling, automated notifications, and smart data maintenance ensures patients receive consistent guidance and support throughout their treatment journey. Ultimately, DoseMate empowers both patients and doctors—improving medication adherence, minimizing missed doses, enhancing healthcare communication, and promoting better treatment outcomes, particularly for vulnerable or underserved populations.

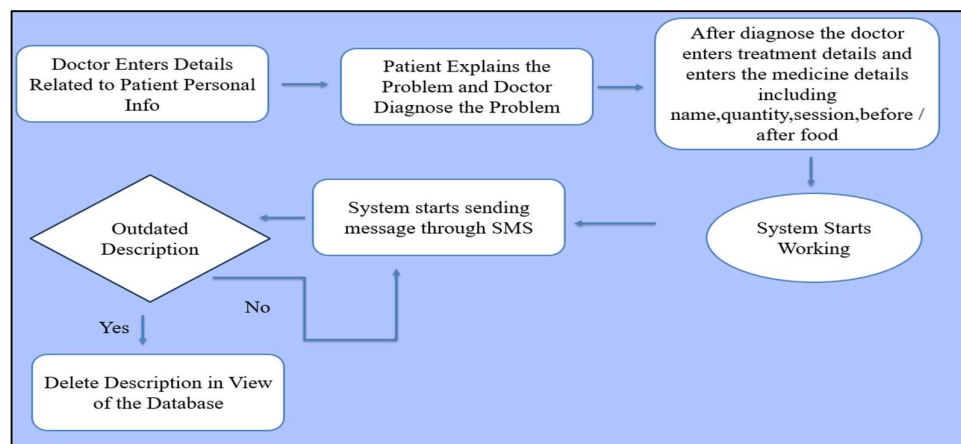


Fig 4.1.1 FLOW DIAGRAM

4.2 BACKEND IMPLEMENTATION

The backend of the DoseMate Healthcare Reminder System is developed using the Spring Boot framework, offering a powerful, modular, and highly reliable foundation for automating patient schedules, managing reminders, and ensuring secure data processing. Spring Boot's microservice-oriented architecture ensures smooth integration between modules, fault tolerance, and high scalability, allowing the system to handle large volumes of data and multiple users simultaneously without performance degradation. Through this backend, doctors can easily input patient details along with the corresponding treatment plans, which include prescribed medicines, injection schedules, lab test dates, and diagnostic scan appointments. Once the information is entered, the backend automatically processes it using a combination of intelligent scheduling and automation algorithms designed to guarantee precision and consistency.

At the core of this automation lie three specialized algorithms — Rule-Based Scheduling, Session-Time Triggering, and Scheduled Expiry Purge — each contributing uniquely to the overall efficiency and reliability of the system.

1. Rule-Based Scheduling Algorithm

The Rule-Based Scheduling Algorithm forms the foundation of reminder generation in DoseMate. It interprets doctor-defined treatment rules and transforms them into specific time-based reminder events. The algorithm operates on logical conditions derived from parameters such as dosage frequency (f), treatment duration (T), and start time (t_0).

The formula for calculating reminder intervals can be expressed as:

$$[t_n = t_0 + T/f * n]$$

where:

(t_n) represents the time for the n th reminder,

(t_0) is the initial reminder start time,

(T) is the total duration of the treatment in hours or days,

(f) is the number of reminders per day, and

(n) denotes the reminder count (1, 2, 3, ...).

This formula ensures that reminders are evenly spaced throughout the day

according to the treatment frequency. For example, if a medication needs to be taken four times a day over a 24-hour period, the system calculates reminders every six hours automatically. These computed intervals are then stored and scheduled by the backend for triggering at the appropriate times.

2. Session-Time Triggering Algorithm

The Session-Time Triggering Algorithm is responsible for dividing the day into four structured sessions — Morning, Afternoon, Evening, and Night — ensuring that reminders are distributed across all key timeframes for better adherence. Each session is associated with two reminders to maintain consistent engagement throughout the day.

Let each session be represented as (S_i) , where:

S_1 =Morning (6 AM – 12 PM), S_2 =Afternoon (12 PM – 6 PM),

S_3 =Evening (6 PM – 10 PM), S_4 =Night (10 PM – 6 AM)

The reminder times for each session are computed using:

$$[R_{i,j} = S_{i,start} + j * (S_{i,end} - S_{i,start}) / N_{i+1}]$$

where:

$(R_{i,j})$ = reminder time for session i and reminder number j ,

$(S_{i,start})$ and $(S_{i,end})$ = start and end times of the session,

(N_i) = number of reminders per session (2 in this case),

(j) = index of the reminder (1 or 2).

This ensures that reminders are evenly distributed within each session. For instance, in the morning session (6 AM–12 PM), reminders may be scheduled at 8 AM and 10 AM automatically.

After the calculation, these reminders are queued into the Spring Boot Scheduler using the `@Scheduled` and `TaskScheduler` modules, which trigger the reminders precisely at the defined timestamps.

Once triggered, the Twilio SMS Notification Service is activated to deliver text messages to patients, reminding them about their medication intake, injections, diagnostic scans, or lab appointments. This automation ensures zero dependency on mobile applications and guarantees message delivery even in low-connectivity regions.

3. Scheduled Expiry Purge Algorithm

To maintain the performance and cleanliness of the database, the Scheduled Expiry Purge Algorithm automatically removes expired reminders and redundant entries daily. This prevents the accumulation of outdated data, optimizes query speed, and maintains efficient backend operation.

The purging process follows the rule:

$$[D_{\text{purge}} = \{ R_i \mid R_{\text{itime}} < T_{\text{current}} \}]$$

where:

(D_{purge}) = dataset of reminders to be deleted,

(R_{itime}) = scheduled time of reminder i ,

(T_{current}) = current system time.

In simple terms, any reminder whose scheduled time has already passed is automatically identified and removed from the system at midnight using Spring Boot's `@Scheduled(cron="0 0 0 * * ?")` annotation. This ensures that the backend remains lightweight, responsive, and efficient even as the number of users increases.

All patient and scheduling data are securely stored in a MySQL database, which ensures data integrity, consistency, and security through constraints, indexing, and transaction management. The system uses JPA (Java Persistence API) for object-relational mapping, ensuring seamless interaction between the application and database. Sensitive information is encrypted, and access is role-based, ensuring compliance with healthcare data protection standards.

Overall, the Spring Boot-based backend of DoseMate integrates these three algorithms seamlessly to provide a scalable, reliable, and intelligent automation framework. It efficiently handles multiple patients, dynamically adjusts to varied schedules, and ensures that every user receives accurate, on-time SMS notifications. This combination of intelligent algorithms and robust architecture establishes DoseMate as a next-generation solution for improving patient adherence, reducing manual workloads, and ensuring effective healthcare management.

4.3 SMS REMINDERS USING TWILIO

DoseMate integrates Twilio, a globally trusted cloud communications platform, to deliver automated SMS notifications that keep patients informed and consistent with their prescribed treatment plans. Twilio serves as the communication backbone of the system, enabling DoseMate to send reliable, real-time reminders directly to patients' mobile phones, regardless of their location or device type. This powerful integration ensures that all patients—whether living in urban centers or remote rural areas—receive timely alerts about their medications, injections, lab tests, and diagnostic appointments without the need for smartphones, internet connectivity, or mobile applications.

Twilio is a cloud-based communication API platform that allows developers to embed messaging, voice, and video functionalities directly into applications. In DoseMate, Twilio's Programmable SMS API is used to send automated messages generated by the backend based on patient schedules and session triggers. With a global delivery network that spans over 180 countries, Twilio ensures that every reminder message is delivered securely and promptly, even in areas with limited network infrastructure. Its high message throughput, delivery confirmation reports, and failover routing mechanisms make it exceptionally reliable for healthcare applications where timeliness is critical.

The DoseMate system divides each day into four primary sessions—morning, afternoon, evening, and night—to ensure comprehensive coverage of a patient's daily schedule. Within each session, two automated reminders are sent via Twilio, ensuring that patients are consistently notified throughout the day about their upcoming medications or healthcare tasks. This session-based scheduling is handled by the backend's intelligent algorithms, which calculate exact trigger times and pass the message data to Twilio's API for delivery. The messages typically include concise and clear information such as:

- Medication name and dosage
- Injection or vaccination reminders
- Scheduled lab tests or diagnostic scan timings
- Instructions or follow-up notes from doctors

This automated system eliminates the need for manual follow-ups, reduces dependency on memory, and helps patients adhere more strictly to their prescribed treatment plans.

The backend, developed using Spring Boot, interacts with Twilio's REST API to trigger SMS notifications automatically. Once a reminder time is reached, the backend sends a POST request to Twilio's API endpoint containing the patient's phone number, the reminder message, and the sender ID. Twilio then processes this request through its distributed messaging gateways and delivers the message to the patient's device within seconds. In cases where a message fails to deliver due to temporary network issues, Twilio automatically retries using alternate routes, ensuring maximum reliability and near-100% delivery success rates.

To maintain system performance and prevent clutter, expired reminders are purged daily through DoseMate's automated data-cleaning mechanism. This ensures that only active reminders are stored in the database, keeping operations smooth and efficient. The entire process—from message generation to delivery and data cleanup—is fully automated, requiring no human intervention once the schedule is set by the doctor.

One of the key advantages of using Twilio is its global scalability and compliance with communication standards. It provides data encryption, message tracking, and audit logging, ensuring that patient data remains secure and private. Furthermore, since Twilio messages are transmitted over standard SMS channels, even patients with basic mobile phones can receive reminders, making the system universally accessible. This inclusivity is especially beneficial for elderly individuals and patients in underdeveloped or rural regions who may lack access to modern smartphones or high-speed internet.

In summary, Twilio plays a crucial role in the success of DoseMate by enabling seamless, automated, and reliable communication between healthcare providers and patients. By combining Twilio's robust messaging infrastructure with DoseMate's intelligent scheduling algorithms, the system ensures that every patient receives timely, consistent, and accurate healthcare reminders—no matter where they are.

4.4 FRONTEND IMPLEMENTATION

The frontend of DoseMate is built using React.js, a powerful and widely adopted JavaScript library known for creating dynamic, efficient, and highly responsive user interfaces. This modern frontend framework provides doctors with an intuitive and user-friendly dashboard for managing patient treatment schedules with minimal effort. The design emphasizes simplicity and efficiency, allowing healthcare professionals to quickly add, edit, and monitor patient details without requiring extensive technical knowledge. Every aspect of the interface is designed to enhance usability and streamline daily healthcare operations.

Through the DoseMate interface, doctors can easily input comprehensive patient details, including name, contact number, diagnosis, and treatment-related data. Once the basic information is entered, doctors can schedule multiple healthcare tasks such as prescription medicines, injections, diagnostic scans, lab tests, and follow-up appointments. Each entry is processed intelligently by the system, which automatically calculates the timing for reminders and ensures that messages are sent at the appropriate intervals. This automation significantly reduces manual workload for medical staff, allowing them to focus more on patient care rather than routine scheduling activities.

React's component-based architecture plays a key role in the frontend's structure and performance. Individual components handle specific functionalities such as patient registration, task scheduling, viewing upcoming reminders, and monitoring system performance. For example, one component may display a list of patients with their corresponding reminder schedules, while another provides real-time updates about the next scheduled SMS notifications. This modular design ensures code reusability, faster rendering, and easier maintenance while keeping the interface highly organized and visually appealing.

To enhance user experience, the frontend is designed with responsive layouts that adapt seamlessly across various devices, including desktops, tablets, and mobile screens. Whether a doctor accesses the application from a hospital computer or a personal tablet, the interface maintains its clarity, accessibility, and functionality. Advanced UI styling libraries and frameworks—such as Tailwind CSS or Material

UI—can be integrated to ensure consistency in color themes, typography, and interactive animations, providing a professional and visually engaging experience.

The frontend communicates directly with the Spring Boot backend through secure RESTful APIs. When doctors input patient data or schedule a new reminder, the frontend sends a structured JSON request to the backend. The backend then processes this information, applies its scheduling algorithms, and stores it in the MySQL database. This tight integration ensures real-time synchronization between the interface and the backend, allowing doctors to instantly view updates such as newly created reminders, modified schedules, or completed tasks without needing to refresh the page.

A key highlight of the frontend is its integration with the Twilio API, which enables doctors to send automated SMS reminders directly from the interface. Once the doctor finalizes the schedule, the system triggers a backend process that uses Twilio's cloud messaging service to deliver SMS notifications to patients. This process is fully automated, meaning the doctor doesn't have to manually send messages or monitor delivery status. Additionally, confirmation messages or delivery reports can be displayed in the frontend dashboard, giving doctors visibility into which reminders have been successfully sent.

To further enhance usability, the interface includes visual indicators and progress tracking features. Doctors can easily view which reminders are active, which have expired, and which are pending for the day. A system status panel provides an overview of all running operations—such as scheduled reminders, database synchronization, and Twilio message delivery logs—helping users ensure that the system is functioning correctly at all times.

In terms of performance, React's virtual DOM (Document Object Model) ensures fast updates and smooth user interaction. This allows DoseMate to handle multiple real-time data changes, such as updates in reminder times or patient additions, without reloading the entire page. The result is a highly efficient, real-time, and interactive user experience that aligns perfectly with the fast-paced needs of healthcare professionals.

In summary, the React.js frontend of DoseMate delivers a seamless, interactive, and scalable user experience tailored for doctors. It simplifies the process of managing patient schedules, automates complex reminder handling, and ensures smooth communication between patients and doctors through Twilio integration. The combination of responsive design, real-time data handling, and backend synchronization ensures that DoseMate remains reliable, efficient, and user-friendly across all devices. Ultimately, the frontend serves as the bridge that connects healthcare professionals to a fully automated, intelligent reminder ecosystem, promoting improved adherence, better patient outcomes, and a smoother workflow for doctors.

4.5 SUMMARY AND INSIGHTS

The DoseMate system is a smart, automated healthcare reminder solution designed to improve patient adherence through session-based SMS notifications. It allows doctors to schedule medicines, injections, lab tests, and diagnostic appointments efficiently. Each day is divided into four key sessions—morning, afternoon, evening, and night—ensuring full-day coverage of patient reminders. Every session triggers two timely alerts, keeping patients informed about their scheduled healthcare activities. This structured and automated approach minimizes the chances of missed doses or appointments, improving treatment consistency and patient health outcomes.

On the backend, DoseMate is powered by Spring Boot, which ensures scalability, robustness, and seamless data handling. Doctors can securely store patient information and schedules in a MySQL database, where the system's algorithms—such as Rule-Based Scheduling, Session-Time Triggering, and Scheduled Expiry Purge—manage and automate reminder generation. Rule-Based Scheduling determines when each reminder should be sent based on the patient's treatment plan, while Session-Time Triggering activates the reminder precisely at predefined intervals. The Scheduled Expiry Purge algorithm removes outdated reminders daily, optimizing performance and maintaining database integrity.

The frontend, developed using React.js, provides an interactive and intuitive interface for doctors. The design allows doctors to effortlessly input patient details, manage schedules, and monitor upcoming reminders. React's component-based architecture ensures fast updates, responsiveness, and smooth user experience across all devices. It also supports real-time communication with the backend, enabling doctors to view live data such as patient statuses, scheduled reminders, and completion reports. The interface integrates seamlessly with backend APIs to ensure accurate data synchronization between the frontend and the server.

The Twilio API integration plays a central role in ensuring that reminders are effectively delivered to patients across the globe. Through Twilio's highly reliable and scalable messaging infrastructure, DoseMate sends SMS reminders even to patients in rural areas with limited internet access. This ensures inclusivity, as patients do not need smartphones or mobile apps to receive notifications. Twilio guarantees high message delivery rates and supports international communication standards. Combined with AWS cloud deployment, DoseMate ensures 24/7 system availability, secure data handling, and effortless scalability. Overall, the system delivers a robust, automated, and globally accessible solution to enhance patient engagement and healthcare compliance.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 SYSTEM SPECIFICATION

The implementation of DoseMate requires reliable hardware to ensure smooth operation and uninterrupted performance. The system efficiently handles multiple patient schedules, automated reminder dispatches, and backend processing. It manages session-based reminders for medicines, injections, lab tests, and other healthcare tasks seamlessly. Reliable hardware ensures that reminders are sent on time across all sessions. The system is capable of supporting simultaneous operations for multiple users. Overall, the hardware provides a stable foundation for DoseMate's functionality.

The software stack is designed to provide a secure, scalable, and efficient environment. Spring Boot powers the backend, managing patient data, schedules, and reminder automation, while React.js offers an intuitive frontend interface for doctors. MySQL ensures safe and organized storage of patient and schedule information. Twilio API is used for sending automated SMS reminders for all sessions and tasks. Hosting on AWS cloud infrastructure allows global access, scalability, and continuous uptime.

5.2 SYSTEM IMPLEMENTATION

The implementation of DoseMate involved a seamless integration of multiple core components, including advanced backend scheduling algorithms, an intuitive frontend interface, a reliable SMS notification system, and secure cloud deployment for uninterrupted operation. The backend, developed using Spring Boot, manages all the logic behind scheduling, timing, and automation. Algorithms such as Rule-Based Scheduling, Session-Time Triggering, and Scheduled Expiry Purge work together to ensure accurate reminder generation and efficient system performance.

Meanwhile, the frontend, built with React.js, provides doctors with an easy-to-use interface to input patient details, assign treatment schedules, and monitor overall progress. This combination of backend intelligence and frontend usability creates a highly efficient and user-friendly healthcare reminder system.

Once the doctor enters patient information and prescribes tasks such as medicines, injections, lab tests, or diagnostic appointments, the system automatically triggers reminders without requiring manual intervention. Each day is divided into four key sessions—morning, afternoon, evening, and night—and two reminders are sent per session to ensure that patients never miss an important healthcare activity. These reminders are delivered through Twilio’s SMS API, which guarantees high message delivery rates across all regions, including remote areas without internet access. The entire system is hosted on AWS Cloud, ensuring scalability, data security, and continuous availability. Overall, the implementation of DoseMate represents a well-orchestrated blend of technology and healthcare, enabling reliable, automated, and accessible patient adherence management.

5.2.1 BACKEND SCHEDULING AND REMINDERS

The backend of DoseMate is designed to provide a highly automated and intelligent reminder delivery system through the integration of three core algorithms — Rule-Based Scheduling, Session-Time Triggering, and Scheduled Expiry Purge. These algorithms work in harmony to ensure timely and precise notifications for every patient. The Rule-Based Scheduling Algorithm determines reminder timings based on the doctor’s prescribed treatment plan, analyzing dosage intervals, medication frequency, and preferred session times. The Session-Time Triggering Algorithm then activates these reminders within the defined four daily sessions — morning, afternoon, evening, and night — sending out two alerts per session. This structured approach ensures consistent notifications for medications, injections, and diagnostic or lab appointments. All patient data, treatment schedules, and session details are securely stored in a MySQL database, ensuring data integrity and quick retrieval whenever required.

To enhance communication reliability, the backend interacts with the Twilio SMS API, which handles the seamless delivery of reminders to patients globally. This ensures that even patients without smartphones or internet connectivity can receive notifications promptly via SMS. Additionally, the Scheduled Expiry Purge Algorithm automatically removes outdated reminders daily, maintaining optimal system performance and preventing unnecessary data accumulation. The backend operates within a Spring Boot framework, offering scalability, modularity, and high-speed processing for multiple concurrent users. By combining automation, security, and efficiency, the DoseMate backend establishes a dependable scheduling framework that minimizes manual involvement while ensuring that every patient receives accurate, on-time reminders to improve adherence and treatment outcomes.

5.2.2 FRONTEND INTERFACE USING REACT.JS

The frontend of DoseMate, built using React.js, provides an intuitive and user-friendly interface designed specifically to simplify healthcare management for doctors. It allows doctors to effortlessly add and manage patient details, prescribe medicines, schedule injections, and set up reminders for lab tests and diagnostic appointments. Each task can be organized across the four daily sessions — morning, afternoon, evening, and night — ensuring complete flexibility and precision in scheduling. The responsive design ensures that the interface adapts smoothly across devices, enabling doctors to monitor schedules and reminders conveniently from desktops, tablets, or mobile devices. Through clearly structured components and dynamic layouts, doctors can easily navigate between patient profiles, view upcoming reminders, and manage multiple treatment schedules simultaneously, reducing administrative workload and the chances of manual oversight.

Furthermore, the frontend seamlessly integrates with the Spring Boot backend through secure API endpoints, ensuring smooth two-way data communication and real-time updates. Whenever a doctor schedules or modifies a reminder, the frontend triggers an automated process that interacts with the backend, which in turn uses Twilio to send timely SMS alerts to patients.

This ensures instant synchronization between the doctor's inputs and patient notifications. The frontend's modular React components make the system scalable, maintainable, and efficient, while the use of state management tools ensures data consistency across the interface. Overall, the React.js frontend of DoseMate enhances usability, ensures real-time responsiveness, and provides a seamless experience that supports doctors in managing patient adherence effectively and efficiently.

5.2.3 SMS DELIVERY USING TWILIO

DoseMate uses the Twilio API to send automated SMS notifications to patients globally. Each session generates two reminders, covering medicines, injections, lab tests, and other healthcare tasks. Twilio ensures reliable message delivery, even for patients with limited access to smartphones. The backend manages the scheduling, sending, and logging of all SMS notifications. Logs provide accountability and allow monitoring of reminder delivery. This system ensures that patients receive timely alerts for all prescribed treatments. Overall, Twilio integration enhances adherence and system reliability.

5.3 RESULTS

The system was tested using sample patient schedules, including medicines, injections, lab tests, and diagnostic appointments across four sessions per day. The system successfully delivered two reminders per session, ensuring patients received timely notifications. SMS delivery via Twilio was consistent and reliable, even for patients without smartphones. Expired reminders were purged automatically to optimize backend performance. The implementation demonstrated that DoseMate improves adherence, reduces missed doses, and supports doctors in managing schedules efficiently. Overall, the system proved scalable, reliable, and effective in enhancing patient compliance.

5.3.1 OUTPUT

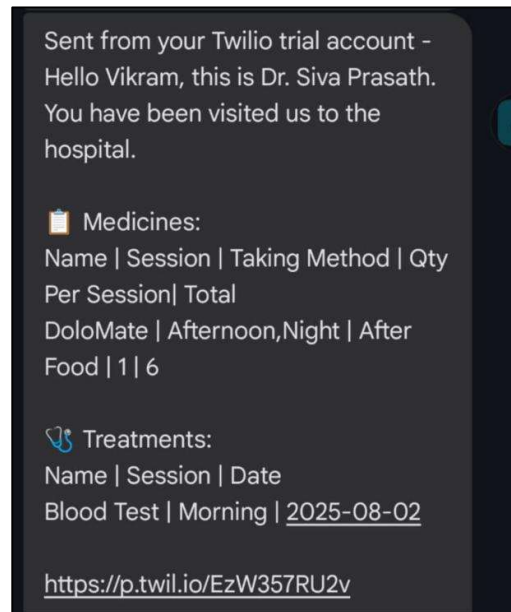


Fig 5.3.1 NEW PATIENT SMS

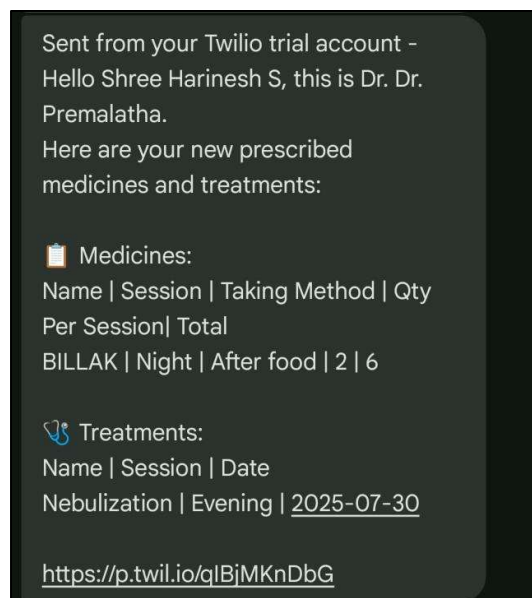


Fig 5.3.2 RETURNING PATIENT SMS

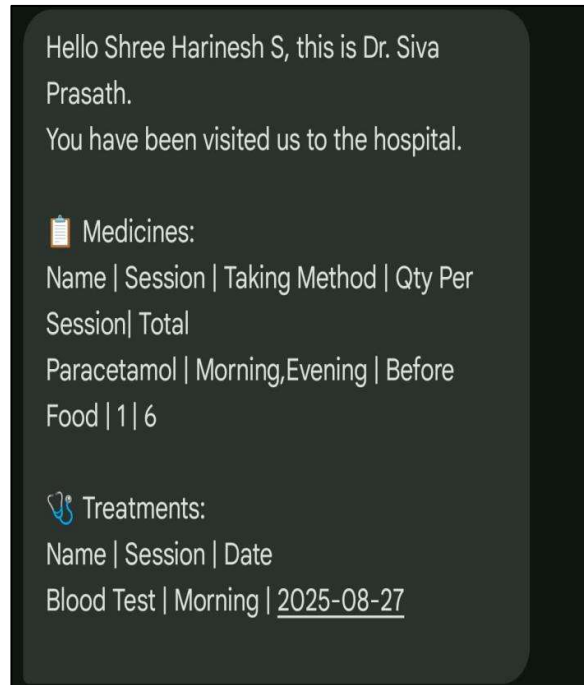


Fig 5.3.3 DIGITAL PRESCRIPTION SMS

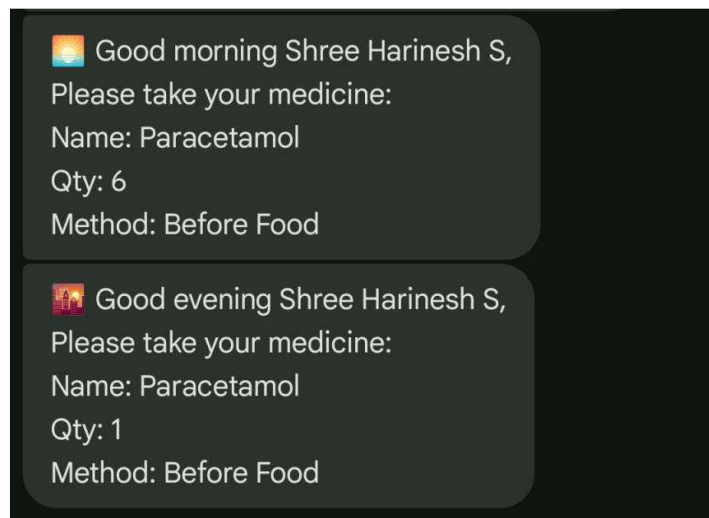


Fig 5.3.4 SESSION MEDICINE REMINDER

Sent from your Twilio trial account - Hello Shree Harinesh S, as doctor advised you to meet him with Blood Test today. Please visit us to keep the health with care.
<https://p.twil.io/pq8MO0RjrG>

Fig 5.3.5 TEST REPORT REMINDER

```

Run DosemateApplication x
Console Beans Health Mappings Environment
SMS sent successfully!
Hibernate: select mr1_0.reminder_id,mr1_0.doctor_name,mr1_0.last_date,mr1_0.medicine_name,mr1_0.patient_id,mr1_0.patient_name,mr1_0.quantity_per_session,mr1_0.session,mr1_0.treatment_name from medicine_reminder mr1_0
Hibernate: select mr1_0.reminder_id,mr1_0.doctor_name,mr1_0.last_date,mr1_0.medicine_name,mr1_0.patient_id,mr1_0.patient_name,mr1_0.quantity_per_session,mr1_0.session,mr1_0.treatment_name from medicine_reminder mr1_0
Hibernate: delete from medicine_reminder where reminder_id=?
Hibernate: select or1_0.reminder_id,or1_0.date_of_reminder,or1_0.doctor_name,or1_0.patient_id,or1_0.patient_name,or1_0.session,or1_0.treatment_name from other_reminder or1_0
Hibernate: select or1_0.reminder_id,or1_0.date_of_reminder,or1_0.doctor_name,or1_0.patient_id,or1_0.patient_name,or1_0.session,or1_0.treatment_name from other_reminder or1_0
Hibernate: delete from other_reminder where reminder_id=?
Reminder cleanup done at 12 AM for both Medicine and Other Treatment Database
  
```

Fig 5.3.6 EXPIRED REMINDER CLEANUP

Admin Login
 Access the administrative dashboard

Email Address

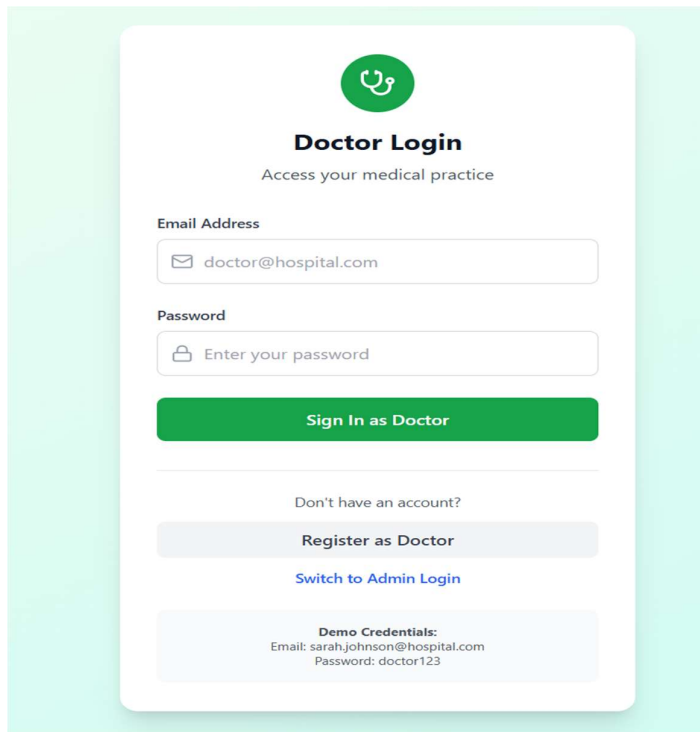
Password

Sign In as Admin

Are you a doctor?
[Switch to Doctor Login](#)

Demo Credentials:
 Email: admin@hospital.com
 Password: admin123

Fig 5.3.7 ADMIN LOGIN



The Doctor Login form features a green circular logo with a white medical symbol at the top. Below the logo, the title "Doctor Login" is displayed in bold, followed by the subtitle "Access your medical practice". The form includes two input fields: "Email Address" with a placeholder "doctor@hospital.com" and "Password" with a placeholder "Enter your password". A green "Sign In as Doctor" button is positioned below the password field. A horizontal line separates the login section from the registration and admin login options. Below the line, the text "Don't have an account?" is followed by a light gray "Register as Doctor" button and a blue "Switch to Admin Login" link. At the bottom, a light gray box contains "Demo Credentials:" with the email "sarah.johnson@hospital.com" and password "doctor123".

Doctor Login
Access your medical practice

Email Address
doctor@hospital.com

Password
Enter your password

Sign In as Doctor

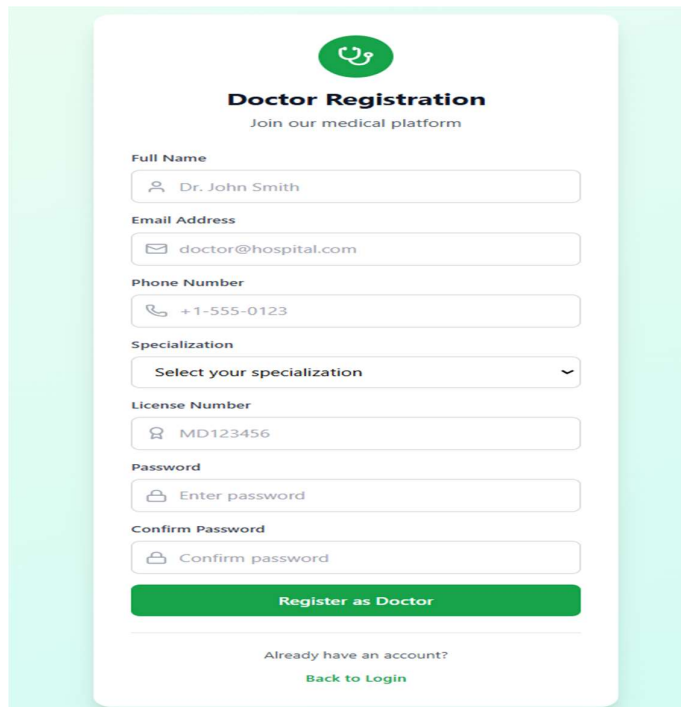
Don't have an account?

Register as Doctor

[Switch to Admin Login](#)

Demo Credentials:
Email: sarah.johnson@hospital.com
Password: doctor123

Fig 5.3.8 DOCTOR LOGIN



The Doctor Registration form features the same green circular logo at the top. Below the logo, the title "Doctor Registration" is displayed in bold, followed by the subtitle "Join our medical platform". The form includes several input fields: "Full Name" with a placeholder "Dr. John Smith", "Email Address" with a placeholder "doctor@hospital.com", "Phone Number" with a placeholder "+1-555-0123", "Specialization" with a dropdown menu labeled "Select your specialization", "License Number" with a placeholder "MD123456", "Password" with a placeholder "Enter password", and "Confirm Password" with a placeholder "Confirm password". A green "Register as Doctor" button is positioned below the password fields. A horizontal line separates the registration section from the login link. Below the line, the text "Already have an account?" is followed by a green "Back to Login" link.

Doctor Registration
Join our medical platform

Full Name
Dr. John Smith

Email Address
doctor@hospital.com

Phone Number
+1-555-0123

Specialization
Select your specialization

License Number
MD123456

Password
Enter password

Confirm Password
Confirm password

Register as Doctor

Already have an account?
[Back to Login](#)

**Fig 5.3.9 DOCTOR
REGISTRATION**

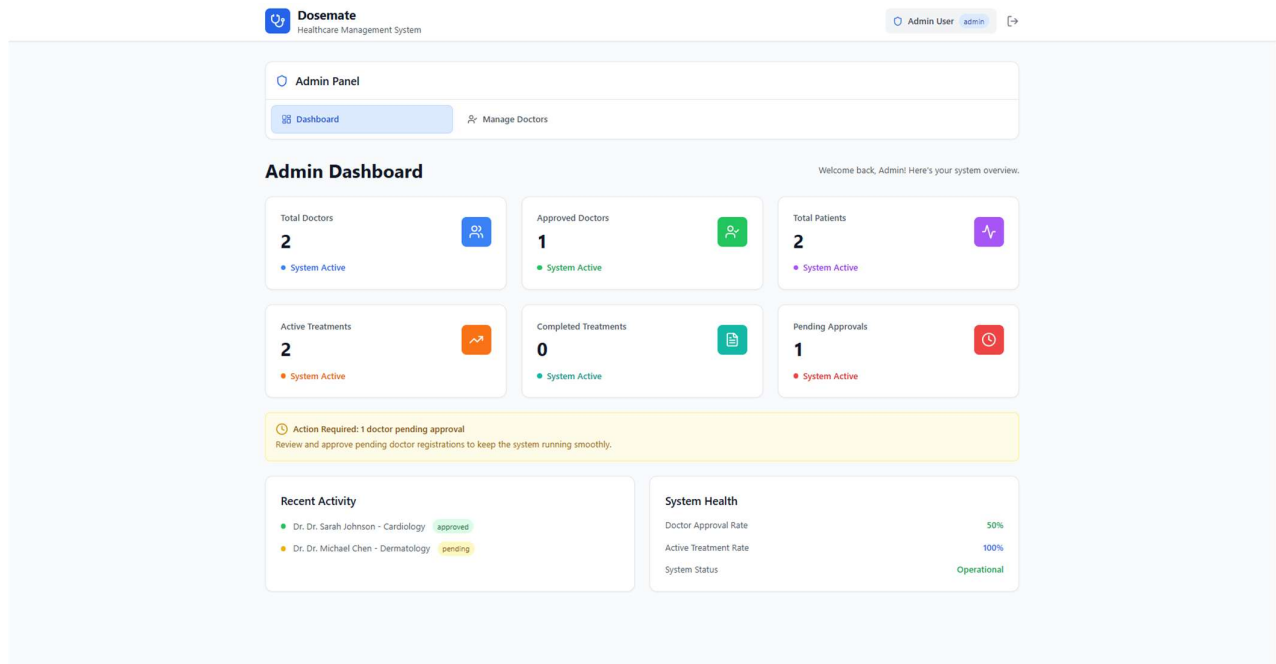


Fig 5.3.10 ADMIN DASHBOARD

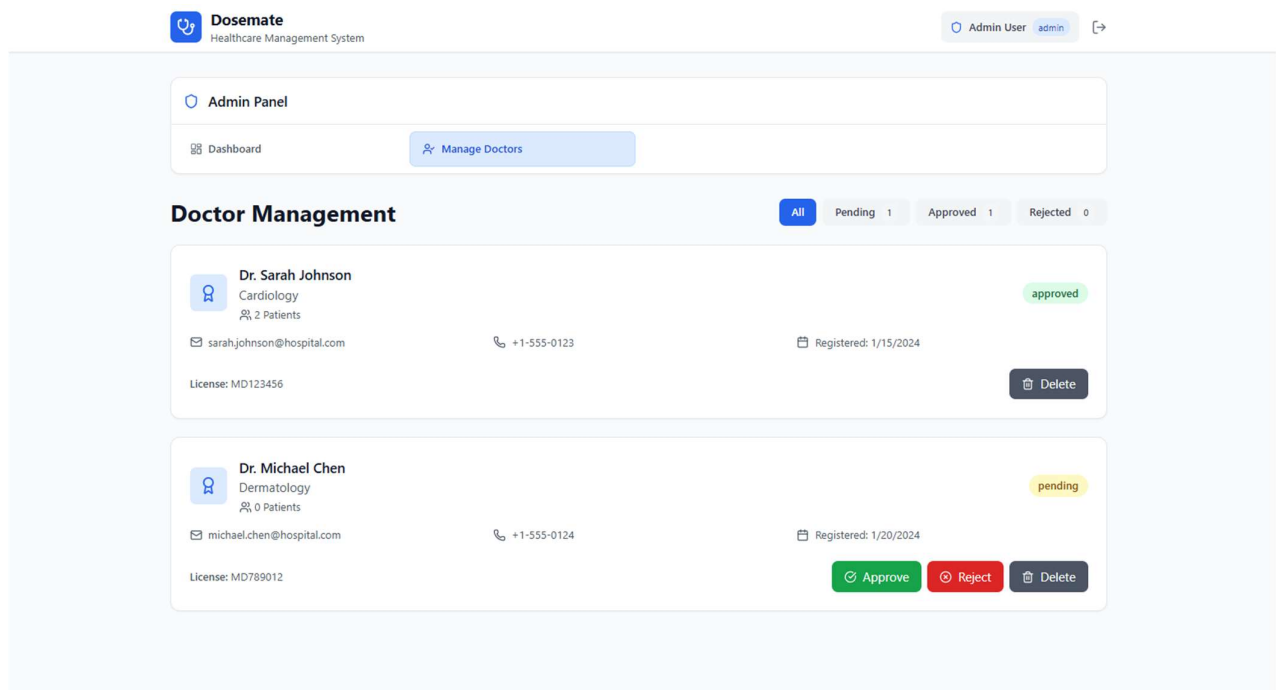


Fig 5.3.11 MANAGE DOCTORS

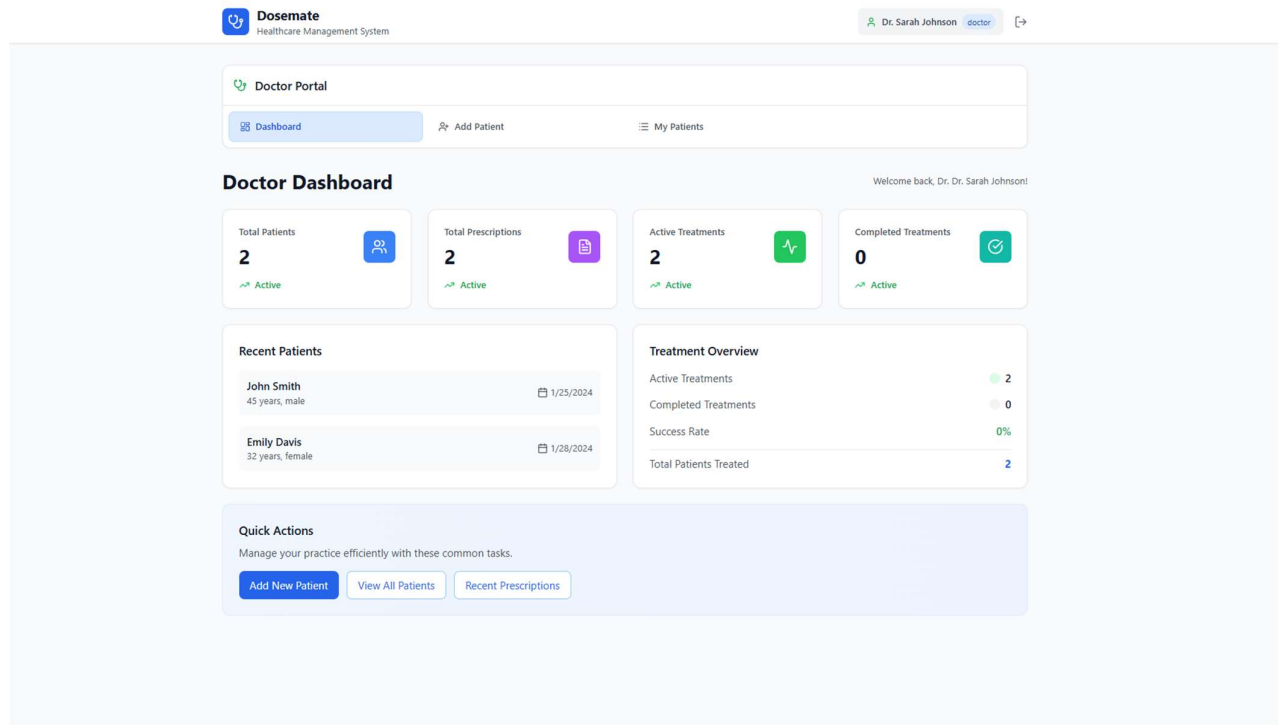


Fig 5.3.12 DOCTOR DASHBOARD

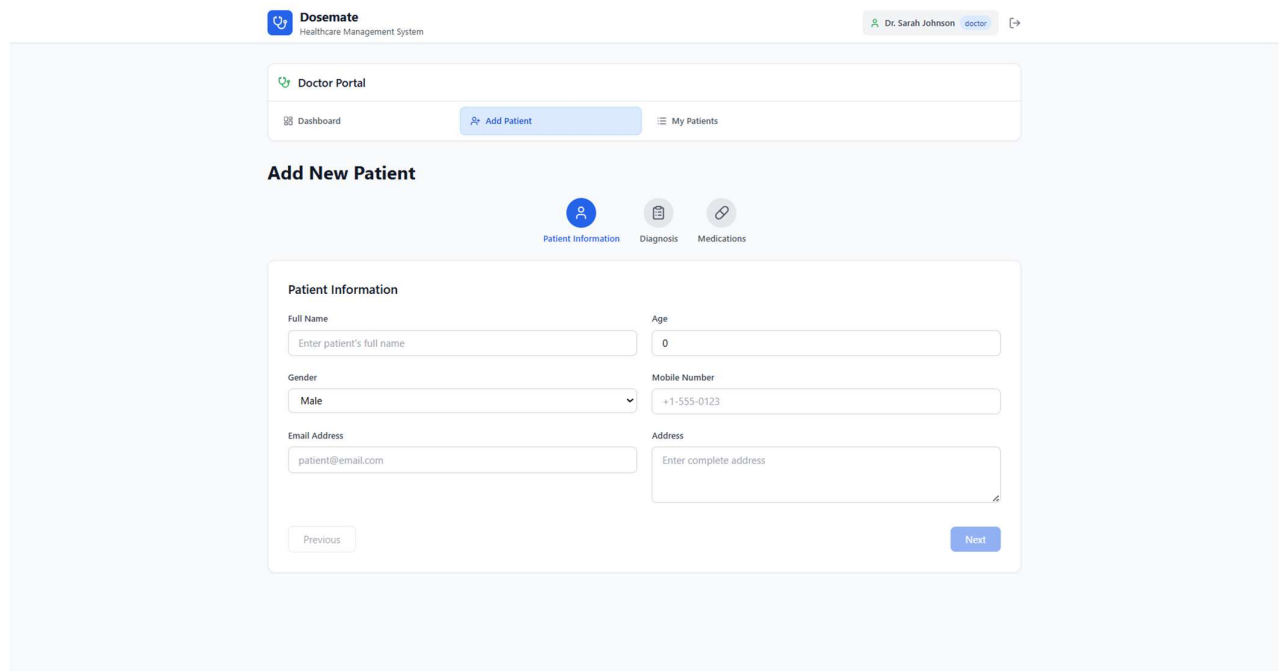


Fig 5.3.13 PATIENT ADDITION

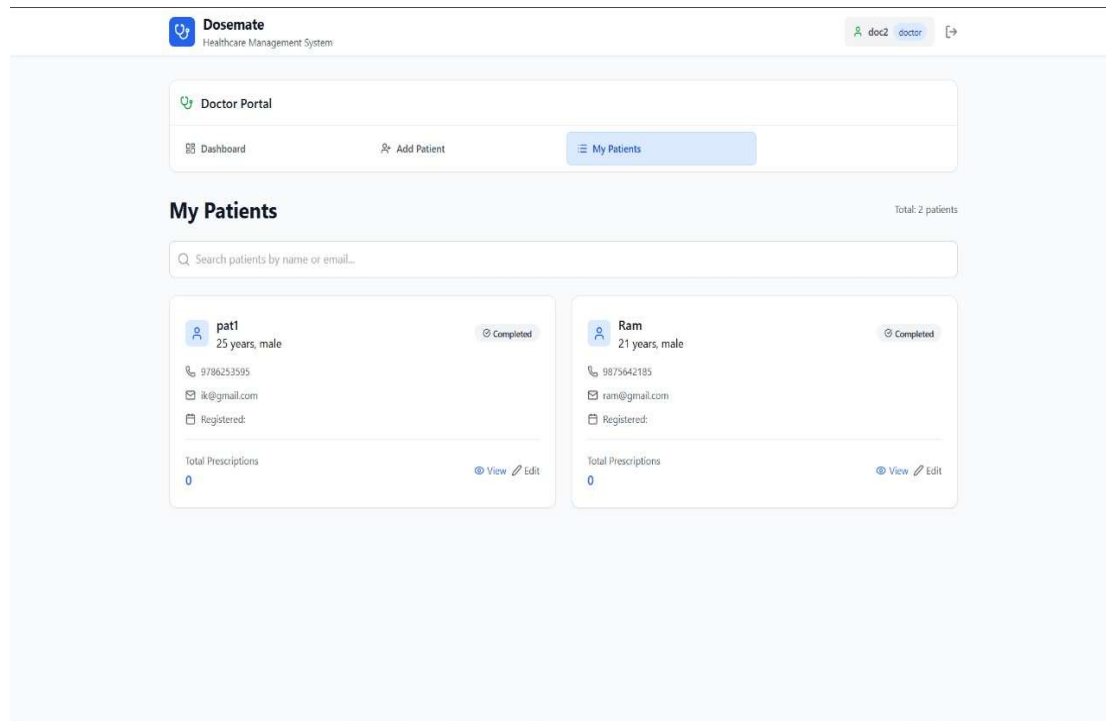


Fig 5.3.14 PATIENTS OVERVIEW

CHAPTER 6

CONCLUSION

6.1 CONCLUSION

The DoseMate healthcare reminder system developed in this project effectively improves patient adherence to prescribed treatments by sending automated, session-based SMS reminders for medicines, injections, lab tests, and other healthcare activities. By leveraging Spring Boot for the backend, React.js for the frontend, Twilio for messaging, and AWS for cloud deployment, the system ensures reliable delivery of reminders and secure management of patient schedules. The use of scheduling algorithms, including Rule-Based Scheduling, Session-Time Triggering, and Scheduled Expiry Purge, guarantees timely notifications and optimized system performance. Overall, DoseMate demonstrates that a technology-driven, scalable reminder system can significantly enhance treatment compliance, reduce missed doses, and support both patients and healthcare providers, particularly in elderly and rural populations.

6.2 FUTURE WORK

While DoseMate currently handles multiple sessions and automated reminders efficiently, there are opportunities for further enhancement. Integration with mobile applications could provide richer interaction and patient feedback. Advanced analytics could help predict adherence patterns and identify high-risk patients. Incorporating additional communication channels such as email or app notifications could complement SMS reminders. Expansion of the system to support multilingual messages would increase accessibility for diverse patient groups. Integration with electronic health records (EHR) systems could streamline data management for healthcare providers. Continuous monitoring and user feedback will be essential to evolve the system and ensure it meets the dynamic needs of patients and doctors worldwide.

REFERENCES

- [1] A. Viljoen, K. Klinker, M. Wiesche, F. Uebernicket, and H. Krcmar, “Design Principles for mHealth Application Development in Rural Parts of Developing Countries: The Case of Noncommunicable Diseases in Kenya,” IEEE, March 2023.
- [2] World Health Organization, “Adherence to Long-Term Therapies: Evidence for Action,” WHO Report, 2021.
- [3] Centers for Disease Control and Prevention, “Improving Medication Adherence Interventions,” CDC Report, 2022.
- [4] Case studies on MDR-TB, HIV, and MRSA, WHO Global Health Data.
- [5] AWS SNS Documentation, “Simple Notification Service for Scalable Messaging,” Amazon Web Services, 2023.
- [6] C. K. Chow et al., “Effect of lifestyle-focused text messaging on risk factor modification in patients with coronary heart disease: a randomized clinical trial,” JAMA, vol. 314, no. 12, pp. 1255–1263, 2015.
- [7] J. Thakkar et al., “Mobile telephone text messaging for medication adherence in chronic disease: a meta-analysis,” JAMA Intern. Med., vol. 176, no. 3, pp. 340–349, 2016.
- [8] D. Arambepola et al., “Effectiveness of mobile phone text messaging in improving medication adherence in chronic diseases: a systematic review,” J. Med. Internet Res., vol. 18, no. 1, p. e25, 2016.
- [9] P. Vervloet et al., “SMS reminders improve adherence to oral medication in type 2 diabetes patients who are realtime electronically monitored,” Diabetes Care, vol. 35, pp. 2537–2539, 2012.
- [10] E. J. van Heuckelum et al., “Short- and long-term effects of real-time medication monitoring with SMS reminders for missed doses on refill adherence of people with Type 2 diabetes,” Diabet. Med., vol. 31, no. 7, pp. 821–828, 2014.
- [11] J. R. MacLean et al., “Text Message Medication Adherence Reminders Automated and Delivered at Scale Across Two Institutions: Testing the Nudge System,” Circ.: Cardiovasc. Qual. Outcomes, vol. 14, no. 6, e007015, 2021.

- [12] T. Alemayehu et al., “Intention to use mobile text message reminders for medication adherence among hypertensive patients in North West Ethiopia: a cross-sectional study,” *BMC Health Serv. Res.*, vol. 24, no. 1, 2024.
- [13] C. Free et al., “The effectiveness of mobile-health technologies to improve health care service delivery processes: a systematic review and meta-analysis,” *PLoS Med.*, vol. 10, no. 1, e1001363, 2013.
- [14] S. Agarwal et al., “Guidelines for reporting of health interventions using mobile phones: mobile health (mHealth) evidence reporting and assessment (mERA) checklist,” *BMJ*, vol. 352, i1174, 2016.
- [15] F. Reiners et al., “Socio-demographic factors influencing the use of eHealth in people with chronic diseases,” *Int. J. Environ. Res. Public Health*, vol. 16, no. 4, p. 645, 2019.
- [16] S. M. S. Islam et al., “Application of mobile phone technology for health monitoring in developing countries: a review,” *J. Biomed. Inform.*, vol. 68, pp. 114–127, 2017.
- [17] R. K. Barik et al., “SoA-Fog: Secure Service-Oriented Edge Computing Architecture for Smart Health Big Data Analytics,” *arXiv preprint arXiv:1712.09098*, 2017.
- [18] Q. Wu et al., “FedHome: Cloud-Edge based Personalized Federated Learning for In-Home Health Monitoring,” *arXiv preprint arXiv:2012.07450*, 2020.
- [19] A. Hennebelle et al., “SmartEdge: Smart Healthcare End-to-End Integrated Edge and Cloud Computing System for Diabetes Prediction,” *arXiv preprint arXiv:2502.15762*, 2025.
- [20] I. Hochberg et al., “A Reinforcement Learning System to Encourage Physical Activity in Diabetes Patients,” *arXiv preprint arXiv:1605.04070*, 2016.
- [21] M. N. K. Boulos et al., “How smartphones are changing the face of mobile and participatory healthcare: an overview,” *Biomed. Eng. Online*, vol. 10, no. 1, 2011.
- [22] R. Agarwal et al., “The digital transformation of healthcare: current status and the road ahead,” *Inf. Syst. Res.*, vol. 21, no. 4, pp. 796–809, 2010.
- [23] J. Kvedar, M. J. Coye, and W. Everett, “Connected health: a review of technologies and strategies to improve patient care with telehealth and telemedicine,” *Health Affairs*, vol. 33, no. 2, pp. 194–199, 2014.

APPENDIX 1 SOURCE CODE

ReminderCleanUpService.java:

```

package com.dosemate.Dosemate.service;

import com.dosemate.Dosemate.model.MedicineReminder;
import com.dosemate.Dosemate.model.OtherReminder;
import com.dosemate.Dosemate.repo.MedicineReminderRepository;
import com.dosemate.Dosemate.repo.OtherReminderRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import java.time.LocalDate;
import java.util.List;

@Service
public class ReminderCleanupService
{
    @Autowired
    private MedicineReminderRepository medicineReminderRepository;
    @Autowired
    private OtherReminderRepository otherTreatmentReminderRepository;
    // Runs daily at 10:00 PM
    @Scheduled(cron = "0 49 10 * * *")
    public void cleanUpExpiredReminders()
    {
        LocalDate today = LocalDate.now();
        // Delete expired medicine reminders
        List<MedicineReminder> expiredMedicines =
medicineReminderRepository.findByLastDateBefore(today);
        medicineReminderRepository.deleteAll(expiredMedicines);
    }
}

```



```
// Delete expired other treatment reminders
    List<OtherReminder> expiredOthers =
otherTreatmentReminderRepository.findByDateOfReminderBefore(today);
    otherTreatmentReminderRepository.deleteAll(expiredOthers);
    System.out.println("Reminder cleanup done at 12 AM for both Medicine and Other
Treatment Database");
}
}
```

ReminderScheduler.java:

```
package com.dosemate.Dosemate.service;
import com.dosemate.Dosemate.model.MedicineReminder;
import com.dosemate.Dosemate.repo.MedicineReminderRepository;
import com.dosemate.Dosemate.repo.PatientInfoRepository;
import com.dosemate.Dosemate.service.SmsSender;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import java.time.LocalDateTime;
import java.util.List;
@Service
public class ReminderScheduler {
    @Autowired
    private MedicineReminderRepository reminderRepo;
    @Autowired
    private SmsSender smsSender;
    @Autowired
    private PatientInfoRepository patientRepo;
    // Map the time slot to session and greeting
    private void sendReminders(String sessionKeyword, String greeting) {
        List<MedicineReminder> reminders =
reminderRepo.findBySessionContainingIgnoreCase(sessionKeyword);
```

```

for (MedicineReminder reminder : reminders) {
    String message = greeting + " " + reminder.getPatientName() + ",\n"
        + "Please take your medicine:\n"
        + "Name: " + reminder.getMedicineName() + "\n"
        + "Qty: " + reminder.getQuantityPerSession() + "\n"
        + "Method: " + reminder.getTakingmethod();
    var patientOpt = patientRepo.findById(reminder.getPatientId());
    if (patientOpt.isPresent()) {
        String phoneNo = patientOpt.get().getPhoneNo();
        System.out.println(phoneNo);
        smsSender.sendSms(phoneNo, message);
    }
}

// Morning reminders at 8 AM & 9 AM
@Scheduled(cron = "0 20 14,9 * * *")
public void sendMorningReminders() {
    System.out.println("Send morning reminders");
    sendReminders("morning", " 🌅 Good morning");
}

// Afternoon reminders at 12 PM & 1 PM
@Scheduled(cron = "0 22 14,13 * * *")
public void sendAfternoonReminders() {
    sendReminders("afternoon", " 🌻 Good afternoon");
}

// Evening reminders at 6 PM & 7 PM
@Scheduled(cron = "0 53 14,19 * * *")
public void sendEveningReminders() {
    sendReminders("evening", " 🌃 Good evening");
}

```

```
// Night reminders at 8 PM & 9 PM
@Scheduled(cron = "0 55 14,21 * * *")
public void sendNightReminders() {
    sendReminders("night", " 🌙 Good night");
}
}
```

SmsReminder.java:

```
@Service
public class SmsSender {
    private final List<TwilioAccount> twilioAccounts = new ArrayList<>();
    private final int MAX_RETRIES = 3;
    public SmsSender() {
        twilioAccounts.add(new TwilioAccount("AC9f4632fdbea7b0d2d79aa7f592e4c086",
"SK171f04b821484f15370f3e58b9805d56", "0ZaaKQln79Przbn37KjBwFFHQcpRyFiH",
"+15206524473")); // Primary
        twilioAccounts.add(new TwilioAccount("AC8e2d4fa6e8593ef6f575c8165c05ebb5",
"SKafe79e4741aad3d3e0ea94253ed605b2", "DWAPkdIg9EY07ZdmoG5mc9ypBz9BfvIg",
"+17178835745"));
        twilioAccounts.add(new TwilioAccount("AC8c9d192b5b159caa7b36c0f68c314cb9",
"SK9daadf4e8605fb9106ebc26662fef4a9",

"94U1V24FhoeceUeDpaTg00JKxkqc88Rv", "+17752489383"));
        // twilioAccounts.add(new TwilioAccount("SID4", "KEY4", "SECRET4",
"+15206524476"));
    }
    public boolean sendSms(String phoneNumber, String message) {
        return trySendWithAccounts(phoneNumber, message, 0);
    }
}
```

```

private boolean trySendWithAccounts(String phoneNumber, String message, int
accountIndex) {
    if (accountIndex >= twilioAccounts.size()) return false;
    TwilioAccount account = twilioAccounts.get(accountIndex);
    Twilio.init(account.apiKeySid, account.apiKeySecret, account.accountSid);
    try {
        // Send MMS (media message) to bypass SMS length limitation
        Message msg = Message.creator(
            new com.twilio.type.PhoneNumber("+91" + phoneNumber),
            new com.twilio.type.PhoneNumber(account.fromNumber),
            message
        ).setMediaUrl(Arrays.asList(
            URI.create("https://dosemateimagesms.s3.us-east-
1.amazonaws.com/ChatGPT+Image+Jul+10%2C+2025%2C+08_46_25+PM.png")
        )).create();

        if (!msg.getStatus().toString().equalsIgnoreCase("failed")) {
            System.out.println("✅ Sent via Twilio Account #" + (accountIndex + 1));
            return true;
        } else {
            System.out.println("⚠️ Failed via Account #" + (accountIndex + 1) + ", trying
next...");
            return trySendWithAccounts(phoneNumber, message, accountIndex + 1);
        }
    }

    catch (Exception e) {
        System.err.println("❌ Error via Account #" + (accountIndex + 1) + ": " +
e.getMessage());
        return trySendWithAccounts(phoneNumber, message, accountIndex + 1);
    }
}

```

OtherReminderScheduler.java:

```

package com.dosemate.Dosemate.service;
import com.dosemate.Dosemate.model.OtherReminder;
import com.dosemate.Dosemate.repo.OtherReminderRepository;
import com.dosemate.Dosemate.repo.PatientInfoRepository;
import com.dosemate.Dosemate.service.SmsSender;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.stereotype.Service;
import java.time.LocalDate;
import java.util.List;
@Service
public class OtherReminderScheduler {
    @Autowired
    private OtherReminderRepository reminderRepository;
    @Autowired
    private SmsSender smsSender;
    @Autowired
    private PatientInfoRepository patientRepo;
    private static final String TEMPLATE = "Hello %s, as doctor advised you to meet him with
%s today. Please visit us to keep the health with care.";
    // Run at 8:00 AM
    @Scheduled(cron = "0 06 15 * * *")
    public void sendReminderAt8AM() {
        sendReminders();
    }
    @Scheduled(cron = "0 52 14 * * *")
    public void sendReminderAt12_30PM() {
        sendReminders();
    }
}

```

```

// Run at 4:00 PM
@Scheduled(cron = "0 53 14 * * *")
public void sendReminderAt4PM() {
    sendReminders();
}

// Run at 6:00 PM
@Scheduled(cron = "0 55 14 * * *")
public void sendReminderAt6PM() {
    sendReminders();
}

private void sendReminders() {
    List<OtherReminder> reminders =
reminderRepository.findByDateOfReminder(LocalDate.now());
    for (OtherReminder reminder : reminders) {
        String message = String.format(TEMPLATE, reminder.getPatientName(),
reminder.getTreatmentName());
        // You can add patient phone number logic accordingly
        var patientOpt = patientRepo.findById(reminder.getPatientId());
        if (patientOpt.isPresent()) {
            String phoneNo = patientOpt.get().getPhoneNo();
            System.out.println(phoneNo);
            smsSender.sendSms(phoneNo, message);
        }
    }
}
}
}

```

AdminController.java :

```

package com.dosemate.Dosemate.controller;
import com.dosemate.Dosemate.model.Admin;
import com.dosemate.Dosemate.model.PatientInfo;
import com.dosemate.Dosemate.service.AdminService;
import com.dosemate.Dosemate.service.DoctorService;

```

```

import com.dosemate.Dosemate.service.PatientService;;
import org.springframework.http.ResponseEntity;
import java.util.List;

@RestController
@RequestMapping("/admin")
public class AdminController {

    @Autowired
    private AdminService adminService;

    @Autowired
    private DoctorService doctorService;

    @Autowired
    private PatientService patientService;

    @PostMapping("/register")
    public ResponseEntity<Admin> registerAdmin(@RequestBody Admin admin) {
        Admin saved = adminService.registerAdmin(admin);
        return ResponseEntity.ok(saved);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Admin> getAdminById(@PathVariable int id) {
        return adminService.getAdminById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    //  New Endpoint: Approve Doctor
    @PutMapping("/approveDoctor/{doctorId}")
    public ResponseEntity<String> approveDoctor(@PathVariable int doctorId) {
        boolean approved = doctorService.approveDoctor(doctorId);
        if (approved) {
            return ResponseEntity.ok("Doctor approved successfully");
        } else {
            return ResponseEntity.badRequest().body("Doctor not found");
        }
    }
}

```

```

// ✅ Optional: Reject Doctor (delete or keep with rejected flag)
@DeleteMapping("/rejectDoctor/{doctorId}")
public ResponseEntity<String> rejectDoctor(@PathVariable int doctorId) {
    boolean deleted = doctorService.rejectDoctor(doctorId);
    if (deleted) {
        return ResponseEntity.ok("Doctor rejected and removed from the database");
    }
else {
        return ResponseEntity.badRequest().body("Doctor not found");
    }
}

@PostMapping("/login")
public String login(@RequestBody Admin request) {
    boolean isAuthenticated = adminService.login(request.getAdminName(),
request.getPassword());

    if (isAuthenticated) {
        return "Login Successful!";
    } else {
        return "Invalid Admin Name or Password!";
    }
}

// ✅ Get all doctors (approved + not approved)
@GetMapping("/doctors")
public ResponseEntity<?> getAllDoctors() {
    return ResponseEntity.ok(doctorService.getAllDoctors());
}

```



```

// ✅ Get total doctor count
@GetMapping("/doctorCount")
public ResponseEntity<?> getDoctorCount() {
    long totalDoctors = doctorService.getTotalDoctors();
    return ResponseEntity.ok(totalDoctors);
}

// ✅ Get count of approved doctors
@GetMapping("/doctorCount/approved")
public ResponseEntity<?> getApprovedDoctorCount() {
    long approvedCount = doctorService.countByApprovalStatus(true);
    return ResponseEntity.ok(approvedCount);
}

// ✅ Get count of unapproved doctors
@GetMapping("/doctorCount/pending")
public ResponseEntity<?> getPendingDoctorCount() {
    long pendingCount = doctorService.countByApprovalStatus(false);
    return ResponseEntity.ok(pendingCount);
}

@GetMapping("/patientCount")
public ResponseEntity<?> getPatientCount() {
    long totalPatients = patientService.getTotalPatients();
    return ResponseEntity.ok(totalPatients);
}

// ✅ Active patient count
@GetMapping("/activePatientCount")
public ResponseEntity<?> getActivePatientCount() {
    try {
        long count = adminService.getActivePatientCount();
        return ResponseEntity.ok(count);
    } catch (Exception e) {
        return ResponseEntity.status(500).body("Error fetching active patient count");
    }
}

```

```
//  Completed patient count

@GetMapping("/completedPatientCount")
public ResponseEntity<?> getCompletedPatientCount() {
    try {
        long count = adminService.getCompletedPatientCount();
        return ResponseEntity.ok(count);
    } catch (Exception e) {
        return ResponseEntity.status(500).body("Error fetching completed patient count");
    }
}
}
```

DoctorController.java :

```
package com.dosemate.Dosemate.controller;
import com.dosemate.Dosemate.DTO.ChangePasswordRequest;
import com.dosemate.Dosemate.model.Admin;
import com.dosemate.Dosemate.model.Doctor;
import com.dosemate.Dosemate.service.DoctorService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/doctor")
public class DoctorController {

    @Autowired
    private DoctorService doctorService;

    @PostMapping("/register")
    public ResponseEntity<Doctor> registerDoctor(@RequestBody Doctor doctor) {
        doctor.setApproved(false);
        Doctor saved = doctorService.registerDoctor(doctor);
        return ResponseEntity.ok(saved);}
}
```

```

@GetMapping("/{id}")
public ResponseEntity<Doctor> getDoctorById(@PathVariable int id) {
    return doctorService.getDoctorById(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

@PutMapping("/{id}/change-password")
public ResponseEntity<String> changePassword(
    @PathVariable int id,
    @RequestBody ChangePasswordRequest request) {
    boolean success = doctorService.changePassword(id, request.getOldPassword(),
request.getNewPassword());
    if (success) {
        return ResponseEntity.ok("Password updated successfully.");
    } else {
        return ResponseEntity.badRequest().body("Invalid old password or doctor not
found.");
    }
}

@PostMapping("/login")
public String loginDoctor(@RequestBody Doctor doctor) {
    Doctor loggedInDoctor = doctorService.loginDoctor(doctor.getDoctorName(),
doctor.getPassword());
    if (loggedInDoctor != null) {
        return "Login Successfull"; //  Successful login
    }
    else
    {
        return "Invalid Doctor Name or Password!";
    }
}

```

```
// ✅ Get total number of patients treated by a specific doctor
@GetMapping("/{doctorId}/patientCount")
public ResponseEntity<?> getPatientCountByDoctor(@PathVariable int doctorId) {
    long count = doctorService.getPatientCountByDoctor(doctorId);
    return ResponseEntity.ok(count);
}

// ✅ Get Active Patient Count only
@GetMapping("/{doctorId}/activePatientCount")
public ResponseEntity<?> getActivePatientCount(@PathVariable int doctorId) {
    try {
        long count = doctorService.getActivePatientCount(doctorId);
        return ResponseEntity.ok(count);
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}

// ✅ Get Completed Patient Count only
@GetMapping("/{doctorId}/completedPatientCount")
public ResponseEntity<?> getCompletedPatientCount(@PathVariable int doctorId) {
    try {
        long count = doctorService.getCompletedPatientCount(doctorId);
        return ResponseEntity.ok(count);
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}

// ✅ Reject Doctor by ID
@DeleteMapping("/reject/{doctorId}")
public ResponseEntity<String> rejectDoctor(@PathVariable int doctorId) {
    boolean deleted = doctorService.rejectDoctor(doctorId);
}
```

```

if (deleted) {
    return ResponseEntity.ok("Doctor successfully rejected");
} else {
    return ResponseEntity.badRequest().body("Doctor not found");
}
}
}

```

PatientController.java :

```

package com.dosemate.Dosemate.controller;
import com.dosemate.Dosemate.DTO.PatientVisitDTO;
import com.dosemate.Dosemate.model.PatientInfo;
import com.dosemate.Dosemate.repo.PatientInfoRepository;
import com.dosemate.Dosemate.service.PatientService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/patient")
public class PatientController {
    @Autowired
    private PatientService patientService;
    @Autowired
    private PatientInfoRepository repo;
    @PostMapping("/new/{doctorId}")
    public ResponseEntity<PatientInfo> saveNewPatient(@RequestBody PatientInfo patient,
    @PathVariable int doctorId) {
        PatientInfo savedPatient = patientService.saveNewPatient(patient, doctorId);
        return ResponseEntity.ok(savedPatient);
    }
}

```

```

    @GetMapping("/search")
    public ResponseEntity<PatientInfo> getPatientByName(@RequestParam String name) {
        return patientService.getPatientByName(name)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping("/visit/{doctorId}")
    public ResponseEntity<PatientInfo> addVisitToExistingPatient(@RequestBody
PatientVisitDTO visitDTO,@PathVariable int doctorId) {
        PatientInfo updatedPatient =
patientService.addVisitToExistingPatient(visitDTO,doctorId);
        return ResponseEntity.ok(updatedPatient);
    }
}

```

AdminDashboard.jsx :

```

import React from 'react';
import { useApp } from '../context/AppContext';
import { Users, UserCheck, Activity, FileText, TrendingUp, Clock } from 'lucide-react';
const AdminDashboard = () => {
    const { doctors, patients, prescriptions } = useApp();
    const approvedDoctors = doctors.filter(d => d.status === 'approved').length;
    const pendingDoctors = doctors.filter(d => d.status === 'pending').length;
    const activeTreatments = prescriptions.filter(p => p.status === 'active').length;
    const completedTreatments = prescriptions.filter(p => p.status === 'completed').length;

```

```
const stats = [  
  {  
    title: 'Total Doctors',  
    value: doctors.length,  
    icon: Users,  
    color: 'bg-blue-500',  
    textColor: 'text-blue-600'  
  },  
  {  
    title: 'Approved Doctors',  
    value: approvedDoctors,  
    icon: UserCheck,  
    color: 'bg-green-500',  
    textColor: 'text-green-600'  
  },  
  {  
    title: 'Total Patients',  
    value: patients.length,  
    icon: Activity,  
    color: 'bg-purple-500',  
    textColor: 'text-purple-600'  
  },  
  {  
    title: 'Active Treatments',  
    value: activeTreatments,  
    icon: TrendingUp,  
    color: 'bg-orange-500',  
    textColor: 'text-orange-600'  
  },  
]
```

```

{
  title: 'Completed Treatments',
  value: completedTreatments,
  icon: FileText,
  color: 'bg-teal-500',
  textColor: 'text-teal-600'
},
{
  title: 'Pending Approvals',
  value: pendingDoctors,
  icon: Clock,
  color: 'bg-red-500',
  textColor: 'text-red-600'
}
];
return (
  <div className="space-y-6">
    <div className="flex items-center justify-between">
      <h2 className="text-3xl font-bold text-gray-900">Admin Dashboard</h2>
      <div className="text-sm text-gray-600">
        Welcome back, Admin! Here's your system overview.
      </div>
    </div>
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
      {stats.map((stat, index) => {
        const Icon = stat.icon;
        return (
          <div
            key={index}
            className="bg-white rounded-xl shadow-sm border border-gray-200 p-6
            hover:shadow-md transition-shadow"
          >

```



```

<div className="flex items-center justify-between">
  <div>
    <p className="text-sm font-medium text-gray-600">{stat.title}</p>
    <p className="text-3xl font-bold text-gray-900 mt-2">{stat.value}</p>
  </div>
  <div className={`$${stat.color} p-3 rounded-lg`} >
    <Icon className="h-6 w-6 text-white" />
  </div>
</div>
<div className="mt-4 flex items-center">
  <div className={`h-2 w-2 rounded-full ${stat.color} mr-2`} ></div>
  <span className={`text-sm ${stat.textColor} font-medium`} >
    System Active
  </span>
</div>
</div>
);
}}
</div>
{pendingDoctors > 0 && (
  <div className="bg-yellow-50 border border-yellow-200 rounded-lg p-4">
    <div className="flex items-center">
      <Clock className="h-5 w-5 text-yellow-600 mr-2" />
      <h3 className="text-sm font-medium text-yellow-800">
        Action Required: {pendingDoctors} doctor{pendingDoctors !== 1 ? 's' : ''}
        pending approval
      </h3>
    </div>
  </div>
)

```

```
<p className="text-sm text-yellow-700 mt-1">
```

```
    Review and approve pending doctor registrations to keep the system running smoothly.
```

```
</p>
```

```
</div>
```

```
)}
```

```
<div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
```

```
  <div className="bg-white rounded-xl shadow-sm border border-gray-200 p-6">
```

```
    <h3 className="text-lg font-semibold text-gray-900 mb-4">Recent Activity</h3>
```

```
    <div className="space-y-3">
```

```
      {doctors.slice(0, 3).map(doctor => (
```

```
        <div key={doctor.id} className="flex items-center space-x-3">
```

```
          <div
```

```
            className={`w-2 h-2 rounded-full ${
```

```
              doctor.status === 'approved'
```

```
                ? 'bg-green-500'
```

```
                : doctor.status === 'pending'
```

```
                ? 'bg-yellow-500'
```

```
                : 'bg-red-500'
```

```
            `}
```

```
          ></div>
```

```
<span className="text-sm text-gray-600">
```

```
    Dr. {doctor.name} - {doctor.specialization}
```

```
</span>
```

```
<span
```

```
  className={`text-xs px-2 py-1 rounded-full ${
```

```
    doctor.status === 'approved'
```

```
      ? 'bg-green-100 text-green-800'
```

```
      : doctor.status === 'pending'
```

```
      ? 'bg-yellow-100 text-yellow-800'
```

```
      : 'bg-red-100 text-red-800'
```

```
    `}>
```

```

{doctor.status}
    </span>
  </div>
  ))}
</div>
</div>
<div className="bg-white rounded-xl shadow-sm border border-gray-200 p-6">
  <h3 className="text-lg font-semibold text-gray-900 mb-4">System Health</h3>
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <span className="text-sm text-gray-600">Doctor Approval Rate</span>
      <span className="text-sm font-medium text-green-600">
        {doctors.length > 0
          ? Math.round((approvedDoctors / doctors.length) * 100)
          : 0}%
      </span>
    </div>
    <div className="flex items-center justify-between">
      <span className="text-sm text-gray-600">Active Treatment Rate</span>
      <span className="text-sm font-medium text-blue-600">
        {prescriptions.length > 0
          ? Math.round((activeTreatments / prescriptions.length) * 100)
          : 0}%
      </span>
    </div>
    <div className="flex items-center justify-between">
      <span className="text-sm text-gray-600">System Status</span>
      <span className="text-sm font-medium text-green-600">Operational</span>
    </div>
  </div>
</div>
</div>
</div>
</div>

```

```
);
};export default AdminDashboard;
```

PatientOverview.jsx :

```
import React, { useState } from 'react';
import { useApp } from '../context/AppContext';
import { User, Phone, Mail, MapPin, Calendar, Search, Filter } from 'lucide-react';
const PatientOverview = () => {
  const { patients, doctors, prescriptions } = useApp();
  const [searchTerm, setSearchTerm] = useState("");
  const [selectedDoctor, setSelectedDoctor] = useState('all');
  const filteredPatients = patients.filter(patient => {
    const matchesSearch =
      patient.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
      patient.email.toLowerCase().includes(searchTerm.toLowerCase());
    const matchesDoctor =
      selectedDoctor === 'all' || patient.doctorId === selectedDoctor;
    return matchesSearch && matchesDoctor;
  });
  const getPatientTreatmentStatus = (patientId) => {
    const patientPrescriptions = prescriptions.filter(p => p.patientId === patientId);
    const activeCount = patientPrescriptions.filter(p => p.status === 'active').length;
    const completedCount = patientPrescriptions.filter(p => p.status === 'completed').length;
    return { active: activeCount, completed: completedCount, total: patientPrescriptions.length
  };
};
const getDoctorName = (doctorId) => {
  const doctor = doctors.find(d => d.id === doctorId);
  return doctor ? doctor.name : 'Unknown Doctor';
};
```

```

return (
  <div className="space-y-6">
    <div className="flex items-center justify-between">
      <h2 className="text-3xl font-bold text-gray-900">Patient Overview</h2>
      <div className="text-sm text-gray-600">
        Total Patients: {patients.length}
      </div>
    </div>
    <div className="flex flex-col sm:flex-row gap-4">
      <div className="relative flex-1">
        <Search className="absolute left-3 top-1/2 transform -translate-y-1/2 h-5 w-5 text-
gray-400" />
        <input
          type="text"
          placeholder="Search patients by name or email..."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
          className="w-full pl-10 pr-4 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500 focus:border-transparent"
        />
      </div>
      <div className="relative">
        <Filter className="absolute left-3 top-1/2 transform -translate-y-1/2 h-5 w-5 text-
gray-400" />
        <select
          value={selectedDoctor}
          onChange={(e) => setSelectedDoctor(e.target.value)}
          className="pl-10 pr-8 py-2 border border-gray-300 rounded-lg focus:ring-2
focus:ring-blue-500 focus:border-transparent bg-white"
        >

```

```

    <option value="all">All Doctors</option>
    {doctors.map(doctor => (
      <option key={doctor.id} value={doctor.id}>{doctor.name}</option>
    ))}
  </select>
</div>
</div>
<div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
  {filteredPatients.map((patient) => {
    const treatmentStatus = getPatientTreatmentStatus(patient.id);
    return (
      <div key={patient.id} className="bg-white rounded-xl shadow-sm border border-
gray-200 p-6">
        <div className="flex items-center justify-between mb-4">
          <div className="flex items-center space-x-3">
            <div className="bg-purple-100 p-2 rounded-lg">
              <User className="h-5 w-5 text-purple-600" />
            </div>
            <div>
              <h3 className="text-lg font-semibold text-gray-900">{patient.name}</h3>
              <p className="text-sm text-gray-600">{patient.age} years,
{patient.gender}</p>
            </div>
          </div>
          <div className="text-right">
            <p className="text-sm font-medium text-gray-900">
              Dr. {getDoctorName(patient.doctorId)}
            </p>
            <p className="text-xs text-gray-600">Attending Physician</p>
          </div>
        </div>
      </div>
    )
  })}
</div>

```

```

<div className="grid grid-cols-1 gap-3 mb-4">
  <div className="flex items-center space-x-2 text-gray-600">
    <Phone className="h-4 w-4" />
    <span className="text-sm">{patient.mobile}</span>
  </div>
  <div className="flex items-center space-x-2 text-gray-600">
    <Mail className="h-4 w-4" />
    <span className="text-sm">{patient.email}</span>
  </div>
  <div className="flex items-center space-x-2 text-gray-600">
    <MapPin className="h-4 w-4" />
    <span className="text-sm">{patient.address}</span>
  </div>
  <div className="flex items-center space-x-2 text-gray-600">
    <Calendar className="h-4 w-4" />
    <span className="text-sm">Registered: {new
Date(patient.registrationDate).toLocaleDateString()}</span>
  </div>
</div>
<div className="border-t border-gray-200 pt-4">
  <h4 className="text-sm font-medium text-gray-900 mb-2">Treatment
Status</h4>
  <div className="grid grid-cols-3 gap-4">
    <div className="text-center">
      <p className="text-lg font-bold text-blue-
600">{treatmentStatus.total}</p>
      <p className="text-xs text-gray-600">Total</p>
    </div>

```

```

<div className="text-center">
    <p className="text-lg font-bold text-green-600">{treatmentStatus.active}</p>
    <p className="text-xs text-gray-600">Active</p>
</div>
<div className="text-center">
    <p className="text-lg font-bold text-gray-600">{treatmentStatus.completed}</p>
    <p className="text-xs text-gray-600">Completed</p>
</div>
</div>
</div>
</div>
);
}}
</div>
{filteredPatients.length === 0 && (
    <div className="text-center py-12">
        <div className="bg-gray-100 rounded-full p-3 w-16 h-16 mx-auto mb-4">
            <User className="h-10 w-10 text-gray-400 mx-auto" />
        </div>
        <h3 className="text-lg font-medium text-gray-900 mb-2">No patients found</h3>
        <p className="text-gray-600">Try adjusting your search criteria or filters.</p>
    </div>
    )}
</div>
);
};
export default PatientOverview;

```


DoctorManagement.jsx :

```

import React, { useState } from 'react';
import { useApp } from '../context/AppContext';
import { CheckCircle, XCircle, Trash2, Mail, Phone, Calendar, Award, Users } from 'lucide-react';

const DoctorManagement = () => {
  const { doctors, setDoctors, patients } = useApp(); // include patients
  const [filter, setFilter] = useState('all');
  const filteredDoctors = doctors.filter(
    doctor => filter === 'all' || doctor.status === filter
  );

  const handleApprove = (doctorId) => {
    setDoctors(doctors.map(doctor =>
      doctor.id === doctorId ? { ...doctor, status: 'approved' } : doctor
    ));
  };

  const handleReject = (doctorId) => {
    setDoctors(doctors.map(doctor =>
      doctor.id === doctorId ? { ...doctor, status: 'rejected' } : doctor
    ));
  };

  const handleDelete = (doctorId) => {
    if (window.confirm('Are you sure you want to delete this doctor?')) {
      setDoctors(doctors.filter(doctor => doctor.id !== doctorId));
    }
  };

  const getStatusColor = (status) => {
    switch (status) {
      case 'approved': return 'bg-green-100 text-green-800';
      case 'rejected': return 'bg-red-100 text-red-800';
      case 'pending': return 'bg-yellow-100 text-yellow-800';
      default: return 'bg-gray-100 text-gray-800';
    }
  }
}

```

```

};
return (
  <div className="space-y-6">
    <div className="flex items-center justify-between">
      <h2 className="text-3xl font-bold text-gray-900">Doctor Management</h2>
      <div className="flex space-x-2">
        {[ 'all', 'pending', 'approved', 'rejected' ].map((status) => (
          <button
            key={status}
            onClick={() => setFilter(status)}
            className={`px-4 py-2 rounded-lg text-sm font-medium transition-colors ${
              filter === status
                ? 'bg-blue-600 text-white'
                : 'bg-gray-100 text-gray-700 hover:bg-gray-200'
            }`}
          >
            {status.charAt(0).toUpperCase() + status.slice(1)}
            {status !== 'all' && (
              <span className="ml-2 bg-white bg-opacity-20 px-2 py-1 rounded-full text-
xs">
                {doctors.filter(d => d.status === status).length}
              </span>
            )}
          </button>
        ))}
      </div>
    </div>

    <div className="grid grid-cols-1 gap-6">
      {filteredDoctors.map((doctor) => {
        const doctorPatients = patients.filter(p => p.doctorId === doctor.id).length; // total
patients

```

```

return (
    <div key={doctor.id} className="bg-white rounded-xl shadow-sm border border-
gray-200 p-6">
    <div className="flex items-center justify-between mb-4">
        <div className="flex items-center space-x-4">
            <div className="bg-blue-100 p-3 rounded-lg">
                <Award className="h-6 w-6 text-blue-600" />
            </div>
            <div>
                <h3 className="text-lg font-semibold text-gray-900">{doctor.name}</h3>
                <p className="text-gray-600">{doctor.specialization}</p>
                <p className="flex items-center text-gray-600 text-sm mt-1">
    <Users className="h-4 w-4 mr-1" /> {doctorPatients} Patient{doctorPatients !== 1 ? 's' : ''}
                </p>
            </div>
        </div>
        <div>
            <span className={`px-3 py-1 rounded-full text-sm font-medium
    ${getStatusColor(doctor.status)} `}>
                {doctor.status}
            </span>
        </div>
    </div>
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
        <div className="flex items-center space-x-2 text-gray-600">
            <Mail className="h-4 w-4" />
            <span className="text-sm">{doctor.email}</span>
        </div>
        <div className="flex items-center space-x-2 text-gray-600">
            <Phone className="h-4 w-4" />
            <span className="text-sm">{doctor.phone}</span>
        </div>
        <div className="flex items-center space-x-2 text-gray-600">

```

```

<Calendar className="h-4 w-4" />
    <span className="text-sm">
        Registered: {new Date(doctor.registrationDate).toLocaleDateString()}
    </span>
</div>
</div>
<div className="flex items-center justify-between">
    <div className="text-sm text-gray-600">
        <span className="font-medium">License:</span> {doctor.licenseNumber}
    </div>
<div className="flex space-x-2">
    {doctor.status === 'pending' && (
        <
            <button
                onClick={() => handleApprove(doctor.id)}
                className="flex items-center space-x-2 bg-green-600 text-white px-4
py-2 rounded-lg hover:bg-green-700 transition-colors"
            >
                <CheckCircle className="h-4 w-4" />
                <span>Approve</span>
            </button>
            <button
                onClick={() => handleReject(doctor.id)}
                className="flex items-center space-x-2 bg-red-600 text-white px-4 py-
2 rounded-lg hover:bg-red-700 transition-colors"
            >
                <XCircle className="h-4 w-4" />
                <span>Reject</span>
            </button>
        </>
    )}

```

```

<button
    onClick={() => handleDelete(doctor.id)}
    className="flex items-center space-x-2 bg-gray-600 text-white px-4 py-2
rounded-lg hover:bg-gray-700 transition-colors"
    >
    <Trash2 className="h-4 w-4" />
    <span>Delete</span>
</button>
</div>
</div>
</div>
);
}}
</div>
{filteredDoctors.length === 0 && (
  <div className="text-center py-12">
    <div className="bg-gray-100 rounded-full p-3 w-16 h-16 mx-auto mb-4">
      <Award className="h-10 w-10 text-gray-400 mx-auto" />
    </div>
    <h3 className="text-lg font-medium text-gray-900 mb-2">No doctors found</h3>
    <p className="text-gray-600">No doctors match the selected filter criteria.</p>
  </div>
  )}
</div>
);
};export default DoctorManagement;

```

Account Dashboard

Develop
Monitor

Messaging (US1)

← Dosemate

Properties
Sender Pool
Link Shortening
Integration
Content Settings
A2P & Compliance
Opt-Out Management

Sender Pool

A Sender represents a channel, number type, or messaging type that can receive and send messages using the Twilio Messaging API. Messaging Services come with [built-in features](#) to help you select the best Sender for your use case and scale your application.

Sender Selection Settings

Add Senders

Select Actions 10 per page

☐	Sender	Friendly Name	Type	Capabilities	Actions
☐	+15206524473	(520) 652-4473	Long Code		Remove

Fig 6.1.1 SENDER POOL

Account Dashboard

Develop
Monitor

Logs

Errors
Messaging
Verify calls
Conferences
Call recordings
Call transcriptions
Video rooms
Video recordings
Video compositions

> Insights
> Alarms

Programmable Messaging Logs

Discover messaging with AI

Search by Message SID Search

Export to CSV

Start Date & Time End Date & Time From To 50 Per Page Filter

DATE	SERVICE	DIRECTION	FROM	TO	BODY	# SEGMENTS	STATUS	MEDIA
2025-11-03 13:28:55 GMT+5:30	-	Outgoing API	(US) +1 5206524473	(IN) +91 9786253595		2	Delivered	
2025-11-03 13:28:05 GMT+5:30	-	Outgoing API	(US) +1 5206524473	(IN) +91 9875642185		3	Delivered	
2025-11-03 11:42:40 GMT+5:30	-	Outgoing API	(US) +1 5206524473	(IN) +91 9786253595		4	Delivered	
2025-11-03 9:47:59 GMT+5:30	-	Outgoing API	(US) +1 5206524473	(IN) +91 9786253595		2	Delivered	
2025-11-03 9:47:54 GMT+5:30	-	Outgoing API	(US) +1 5206524473	(IN) +91 9786253595		3	Delivered	
2025-11-02 19:25:01 GMT+5:30	-	Outgoing API	(US) +1 5206524473	(IN) +91 9940744598		2	Delivered	

Fig 6.1.2 MESSAGE LOGS

Result Grid						
			Filter Rows:		Edit:	Export/Import:
	patient_id	email	name	phone_no	whatsapp_no	doctor_id
▶	1	hari@example.com	Vikram	9385700848	9876543210	1
	2	hari@example.com	Vikram	9385700848	9876543210	1
	3	hari@example.com	Hari	9385700848	9876543210	1

Fig 6.1.3 PATIENT DATA

	reminder_id	doctor_name	last_date	medicine_name	patient_id	patient_name	quantity_per_session	session	start_date	takingmethod	total_quantity
▶	1	Dr. Siva Prasath	2025-10-17	DoloMate	1	Vikram	1	Morning,Night	2025-10-14	After Food	6
	2	Dr. Siva Prasath	2025-10-15	DoloMate	2	Vikram	1	Morning,Night	2025-10-11	After Food	6

Fig 6.1.4 MEDICINE REMIDER SCHEDULER

	lsi_id	date_of_reminder	session	treatment_name
▶	1	2025-08-20	Morning	Blood Test
	2	2025-08-20	Morning	Blood Test
	3	2025-08-20	Morning	Blood Test

Fig 6.1.5 LAB REMINDER SCHEDULER